

Smart Bite



By:

Alishba

38846

Vareesha Naeem

35462

Hira Ejaz

38024

Supervised by:

Ms. Hajra Murtaza

Faculty of Computing

Riphah International University, Islamabad

Spring 2025

A Dissertation Submitted To

Faculty of Computing,

Riphah International University, Islamabad

As a Partial Fulfillment of the Requirement for the Award of

The Degree of

Bachelors of Science in Software Engineering

Faculty of Computing
Riphah International University, Islamabad

Date: 27 May, 2025

Final Approval

This is to certify that we have read the report submitted by ***Alishba (38846)***, ***Hira Ejaz(38024)***, ***Vareesha Naeem (34562)***, for the partial fulfillment of the requirements for the degree of the Bachelors of Science in Software Engineering (BSSE). It is our judgment that this report is of sufficient standard to warrant its acceptance by Riphah International University, Islamabad for the degree of Bachelors of Science in Software Engineering (BSSE).

Committee:

1

Ms. Hajra Murtaza
(Supervisor)

2

Dr. Tahira Nazir
(Head of Department)

Declaration

We hereby declare that this document “**Smart Bite Hub**” neither as a whole nor as a part has been copied out from any source. It is further declared that we have done this project with the accompanied report entirely on the basis of our personal efforts, under the proficient guidance of our teachers especially our supervisor **Ms. Hajra Murtaza**. If any part of the system is proved to be copied out from any source or found to be reproduction of any project from anywhere else, we shall stand by the consequences.

Alishba

38846

Hira Ejaz

38024

Vareesha Naeem

35462

Dedication

With heartfelt gratitude to Allah Almighty for granting us the strength and perseverance to complete this final year project report. Special thanks to our dedicated supervisor, **Ms. Hajra Murtaza**, whose invaluable guidance and support were instrumental throughout the development of **Smart Bite Hub**. We also extend our deepest appreciation to each team member; whose hard work and commitment were essential in bringing this project to life. Your dedication turned challenges into achievements, transforming our ideas into a reality.

Abstract

In today's fast-paced campus environment, managing cafeteria operations efficiently is a significant challenge. The Smart Bite Hub addresses these issues by offering a comprehensive, AI-powered platform for automating cafeteria services. By digitizing the ordering process and feedback collection, Smart Bite Hub creates a seamless and efficient experience for all users. The system is designed to solve inefficiencies such as long queues, longer waiting times, waste of resources, unavailability of food items, and lack of record management. Instead of relying on traditional, manual methods, Smart Bite Hub provides a convenient, digital solution that benefits students, faculty members, and cafeteria administration. Students and faculty members use a mobile app to view the digital menu, place orders, and provide feedback. This ensures that they can easily access the cafeteria services without waiting in queues. The mobile app adapts its interface based on user preferences and historical data, guiding users through a personalized experience. The cafeteria administration interacts with the Smart Bite Hub through a web application. This platform allows them to manage user registrations, track orders in real-time, update menus, and analyze feedback to continuously improve services. The system collects and processes data, providing insights that help the administration optimize resource allocation and enhance service quality. The Smart Bite Hub aims to revolutionize cafeteria management, making it accessible and efficient for all stakeholders. Through feedback integration, and digital convenience, Smart Bite Hub creates a supportive and practical environment, helping users maximize their potential and enjoy a better campus experience.

Acknowledgement

First of all, we are grateful to Allah Almighty, the Merciful, the Beneficent, and the source of all knowledge, for granting us the courage and wisdom to complete this project. We extend our sincere thanks to our supervisor, **Ms. Hajra Murtaza**, whose exceptional guidance enabled us to accomplish this project successfully. Additionally, we are thankful to our dedicated team, whose collective efforts and cooperation played a significant role in making this achievement possible.

Alishba
38846

Hira Ejaz
38024

Vareesha Naeem
35462

Table of Contents

Contents

Chapter 1:	12
Introduction	12
1. Introduction	13
1.1 Opportunity & Stakeholders	13
1.2 Motivation & Challenges.....	13
1.3 Goals & Objectives	14
1.4 Solution Overview.....	14
1.5 Report Outline	15
Chapter 2:	16
Literature/Market Survey	16
2.1 Introduction.....	17
2.2 Literature Review/ Technologies Overview	17
2.3 Market Research	19
2.4 Market Survey.....	19
2.4.1 Sample Questionnaire:	19
2.3 Summary	21
Chapter 3:	22
Requirement Analysis	22
3.1 Introduction.....	23
3.2 Problem Solving	23
Chapter 4:	31
Design Document	31
System Design	32
4.1 Introduction.....	32
4.2 Architectural Design.....	32
<i>Figure 4.1 Basic Level Architectural Design</i>	33
4.3 Detailed Design.....	34
Use Case 1: Signup.....	39
Use Case 2: Faculty Signup.....	40
Use Case 3: Login.....	41
Use Case 4: Place order	43
Use Case 5: Make payment	44
Use Case 6: Cancel order	45
Use Case 7: Manage Menu.....	47
Use Case 8: Recharge Wallet	48
Use Case 9: Manage order	49
5.1 Endeavour (Team + Work + Way of Working)	65
5.2 Flow Control/Pseudo codes.....	67
5.3 Components, Libraries, Web Services, and Stubs	72
5.2 Best Practices / Coding Standards	74
5.3 Deployment Environment	75
5.4 Summary	80
6.1 Introduction.....	82
6.2 List of test scenarios.....	82

6.3 Performance and evaluation.....	82
Chapter 7:.....	83
Conclusion and Outlook.....	83
7.1 Introduction.....	84
7.2 Critical Review.....	84
7.3 Future Recommendations/Outlook.....	84
7.4 Summary.....	84
7.5 Achievements and Improvements	84
Reference and Bibliography.....	85
Appendix A: System Sequence Diagram.....	86
1. Signup	87
2. Login	88
3. Place order.....	89
3. Provide Feedback.....	90
4. Forgot password.....	91
5. Recharge wallet.....	92
6. Review feedback.....	93
7. Manage order	94
8. Manage menu	95
9. Faculty order	96
10. Bulk order.....	97
11. Deliver order.....	98
Appendix B: Activity Diagram.....	99
1. Signup	100
2. Login	101
3. Place order.....	102
4. Provide feedback.....	103
5. Manage order	104
6. Schedule bulk order.....	105
7. Manage records.....	106
8. Schedule Pre-order	107
9. Special menu.....	108
10. Regular menu	109
11. Daily menu.....	110
12. Recharge wallet.....	111
13. Deliver order	112
14. Cancel order	113
Appendix C: Test Scenarios.....	114
1. ECP and BVA:.....	115
Appendix D: Work Breakdown Structure.....	134
Appendix E: Roles & Responsibility Matrix	137
Roles & Responsibility Matrix:	138

List of Figures

Figure 4.2.1 Basic Level Architectural Design.....	33
Figure 4.2.2.1 Detailed Level Architectural Design.....	33
Figure 4.3 Package Design.....	33
Figure 4.3.2.1 Deployment Diagram.....	35
Figure A.1 Signup.....	87
Figure A.2 Login.....	88
Figure A.3 Place order	89
Figure A.4 Provide Feedback	90
Figure A.5 Forgot password	91
Figure A.6 Recharge wallet	92
Figure A.7 Review feedback	93
Figure A.8 Manage order.....	94
Figure A.9 Manage menu.....	95
Figure A.10 Faculty order	96
Figure A.11 Bulk order	97
Figure A.12 Deliver order	98
Figure B.1 Signup	100
Figure B.2 Login	101
Figure B.3 Place order	102
Figure B.4 Provide feedback	103
Figure B.5 Manage order.....	104
Figure B.6 Schedule bulk order	105
Figure B.7 Manage records	106
Figure B.8 Schedule Pre-order	107
Figure B.9 Special menu	108
Figure B.10 Regular menu.....	109
Figure B.11 Daily menu	110
Figure B.12 Recharge wallet.....	111
Figure B.13 Deliver order	112
Figure B.14 Cancel order.....	113

List Of Tables

Table 1.1 Literature Review	18
Table 1.2 Planning	65
Table 1.3 Requirement Gathering	66
Table 1.4 Designing.....	66
Table 1.5 Implementation	66
Table 1.6 Testing.....	67
Table 1.7 Hardware Requirements	76
Table 1.8 Software Requirements	76
Table 1.9 ECP	115
Table 1.10 BVA.....	116
Table 1.11 ECP	116
Table 1.12 BVA.....	117
Table 1.12 ECP	118
Table 1.13 BVA.....	118
Table 1.13 Testcase Criteria	120
Table 1.14 Testcases	133
Table 1.15 WBS.....	136
Table 1.16 Roles & Responsibility Matrix.....	138

Chapter 1:

Introduction

1. Introduction

1.1 Opportunity & Stakeholders

The "Smart Bite Hub" software system is designed to automate the operations of the university cafeteria, addressing inefficiencies in the current manual processes due to long queues, longer waiting time, waste of resources, unavailability of food items, lack of record management. This system offers a more convenient and efficient experience by it enhances the overall experience for all users. Faculty members benefit from the convenience of having orders delivered directly to their workplace, while students and staff enjoy a quicker, more efficient service. The system also collects feedback and suggestions, allowing the cafeteria to continuously improve its services.

Stakeholders include:

- **Students:** Students can order online priorly without waiting in queues.
- **Faculty members:** Teachers can order food online and get it delivered right to workplace which is very convenient.
- **Cafeteria administration:** The administration can easily manage the system and upgrade their quality on reviews. Feedbacks will help the administration to customize the menu according to user needs which prevents the wastage of resources.
- **Delivery person:** The delivery person can check orders, view the map, and deliver food to faculty members for their convenience. To improve delivery efficiency, a map and timer are provided to simplify and streamline deliveries within the campus.

1.2 Motivation & Challenges

The following challenges were observed;

- **Inefficient Ordering:** Long queues, especially during peak hours
- **Inconvenience for Faculty:** Must leave workspaces, disrupting schedules
- **Limited Feedback:** No structured method for collecting user reviews
- **Manual Management:** Inefficient operations with no digital tools
- **Resource wastage:** Excess food due to lack of demand estimating.
- **Payments:** Cash problems.

1.3 Goals & Objectives

The Smart Bite Hub will consist of several key modules aimed at improving the efficiency of cafeteria services and enhancing the experience for all users. These include:

1.3.1 For Administrators:

- **User Management:** Manage user registration and maintain accurate records of all users and transactions.
- **Order & Menu Management:** Track and manage orders in real-time, update digital menus.
- **Feedback Management:** Collect and analyze user feedback to continuously improve services.

1.3.2 For Users:

- **Self-Registration:** Easily sign up and access the system using their phone number or Gmail.
- **Ordering & Payment:** View the digital menu and place orders without waiting in queues.
- **Feedback:** Provide feedback directly through the platform to help enhance service quality.

1.4 Solution Overview

To address these challenges, we propose the implementation of the "**Smart Bite Hub**", Digital Cafeteria Management System. This system will utilize a mobile or web-based platform to digitize the cafeteria's operations, making it more efficient and convenient. Staff and students will be able to access a digital menu and place them from anywhere on campus, significantly reducing wait times. Moreover, the system will be more convenient for cafeteria administration as they will manage resources according to quantity of orders which will save their time and resources.

1.5 Report Outline

1.5.1 Opportunities

"**Smart Bite Hub**" is designed to solve inefficiencies in the university cafeteria's manual operations, like long queues, food unavailability, and resource waste. This system provides convenience for all users: students can order online without waiting, faculty can have food delivered to their workplace, and cafeteria staff can enhance services using structured feedback.

1.5.2 Challenges

Current cafeteria challenges include long wait times, inconvenience for faculty, and limited feedback collection. Manual processes also lead to wasted food due to poor demand tracking, while cash-only payments add to the inconvenience. These issues highlight the need for a digital solution to streamline operations and improve service.

1.5.3 Objectives

"**Smart Bite Hub**" will have key modules for administrators and users. Administrators can manage users, track orders, and collect feedback to enhance services. Users can register, view digital menus, order without queues, and provide feedback to help improve their experience. These features aim to make the cafeteria faster, more organized, and user-friendly.

Chapter 2:

Literature/Market Survey

2.1 Introduction

This reviews similar systems in food ordering services to understand how they work and solve common problems by looking other apps and services that offer features like online ordering, cashless payments, and customer feedback, long wait times, running out of stock, and making things easier for users. This survey is used to design the "Smart Bite Hub" app to better meet the needs of students, faculty, and cafeteria staff.

2.2 Literature Review/ Technologies Overview

Features Id	Feature Name	Riphah Cafeteri a System	Air Cafe	Fast Cafe	UET Cafe	IIUI Cafe	Smart Bite Hub
F-1	Create Account	×	×	×	×	×	×
F-2	Login	×	×	×	×	×	×
F-3	Verify Account	×	×	×	×	×	×
F-4	Update Profile	×	×	×	×	×	×
F-5	Place Digital Order	×	×	×	×	×	×
F-6	View menu	×	×	×	×	×	×
F-7	Make Payment	×	×	×	×	×	×
F-8	Order from Office	×	×	×	×	×	×
F-9	Give feedback	×	×	×	×	×	×

F-10	Manage orders	×	×	×	×	×	×
F-11	Review feedback	×	×	×	×	×	×

F-12	Schedule Pre-orders	×	×	×	×	×	×
F-13	Manage menu	×	×	×	×	×	×
F-14	Deliver group orders	×	×	×	×	×	×
F-15	Cancel order	×	×	×	×	×	×
F-16	Manage Records	×	×	×	×	×	×
F-17	Notification	×	×	×	×	×	×
F-18	Recharge E-wallet	×	×	×	×	×	×

Table 1.1 Literature Review

2.3 Market Research

Market research for Smart Bite Hub involves gathering insights about students, faculty, and cafeteria vendors. It helps identify user needs, eating habits, and payment preferences. Methods include online surveys, interviews, and observing cafeteria behavior. This research ensures the system aligns with user expectations and enhances overall service efficiency.

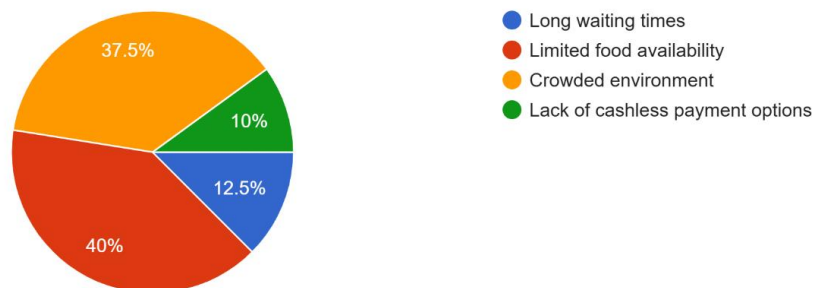
2.4 Market Survey

We conducted a market survey targeting university students and faculty to understand their meal ordering habits, preferred payment methods, and expectations from a digital cafeteria system. The results will guide Smart Bite Hub's features, ensuring a user-friendly experience and improved service satisfaction.

2.4.1 Sample Questionnaire:

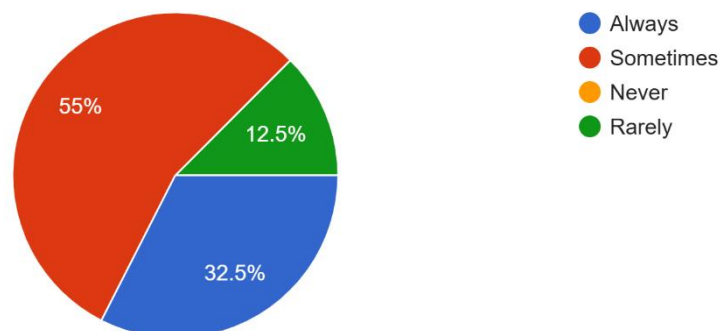
2. What is the biggest challenge you face when ordering food at the cafeteria?

40 responses



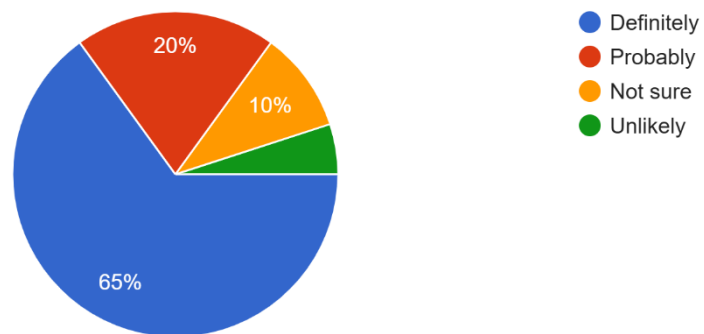
1. How often do you face long waiting times at the cafeteria?

40 responses



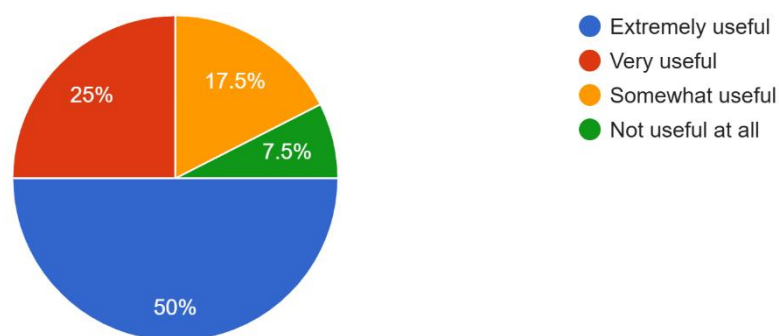
3. Would you use a pre-ordering system to avoid waiting in line at the cafeteria?

40 responses



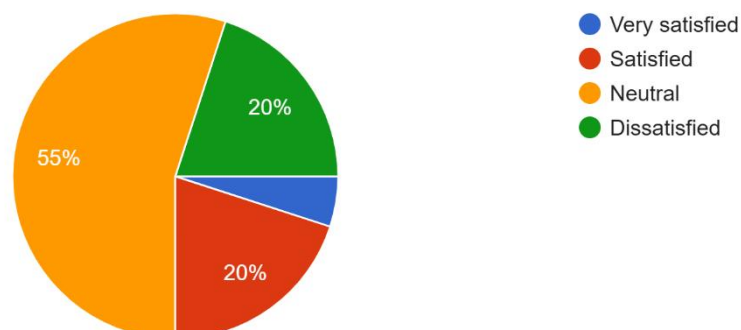
4. How useful do you find the idea of a cashless e-wallet system for paying at the cafeteria?

40 responses



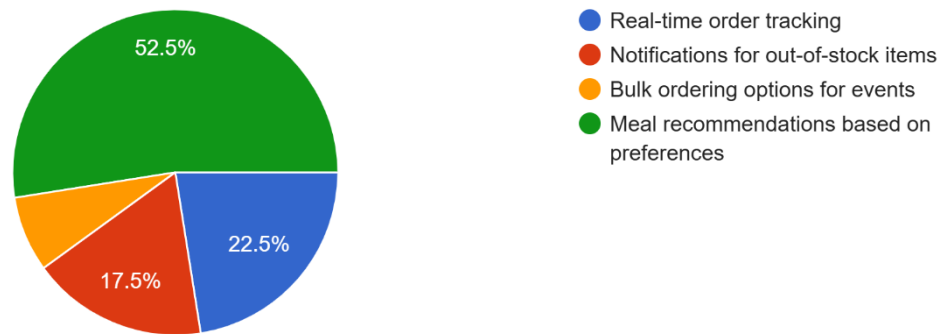
5. How satisfied are you with the current cafeteria system for placing orders and receiving food?

40 responses



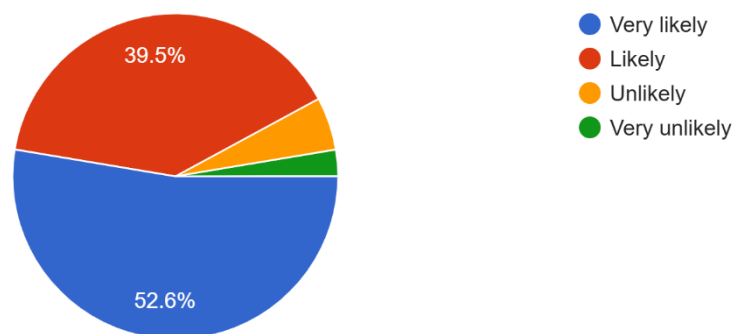
6. What additional feature would you most like to see in Smart Bite Hub?

40 responses



7. If Smart Bite Hub offered a delivery service for faculty, how likely are you to use it?

38 responses



2.3 Summary

In summary, the existing food service systems and digital solutions shows useful features that can improve cafeteria management and user experience. Online ordering, cashless payments, and feedback options help reduce wait times, avoid cash issues, and better manage stock. These surveys highlight the need for user friendly app like "**Smart Bite Hub**" to solve common cafeteria problems effectively.

Chapter 3:

Requirement Analysis

3.1 Introduction

The purpose is to clearly define the project's objectives and outline the essential requirements for creating the "*Smart Bite Hub*" app. This analysis identifies the key challenges and needs for the cafeteria system. It provides a user-friendly solution that will simplify cafeteria operations, allow cashless payments, and help reduce food waste by managing resources more efficiently. Requirement analysis presents a structured overview of the identified problems, along with detailed functional and non-functional requirements, all aimed is to create an efficient system for students, faculty, and administrators alike.

3.2 Problem Solving

The cafeteria currently operates manually and inefficiently, leading to several issues for students, faculty, and staff. During busy hours, the ordering process is slow, causing long wait times. Additionally, popular menu items often run out, leaving customers frustrated when they arrive only to find their choices unavailable. This lack of organization also results in wasted food and resources, as items are sometimes prepared without confirmed orders. For many, carrying cash is inconvenient, and the absence of cashless payment options adds to the hassle. Overall, these issues highlight the need for a more efficient and user-friendly system to manage cafeteria operations smoothly.

3.3 Functional Requirements

3.3.1. Create Account

FR.1. The system shall allow students to create accounts by providing their valid student identification.

FR.2. The system shall allow faculty to create accounts by providing their valid faculty identification.

FR.3. Student and faculty must provide additional personal information such as name and email during the signup process.

FR.4. The system shall allow the admin to verify user details (ID and email) before granting account access.

FR.5. Student and faculty shall be prompted to set a secure password that includes at least 8 characters, with a mix of uppercase, lowercase letters, numbers, and special characters.

3.1.2. Login

FR.1. The system shall allow student and faculty to log in using their registered email and password.

FR.2. Upon successful login, student and faculty shall be directed to a personalized dashboard based on their account type faculty or student.

3.1.3. Place Digital Order

FR.1. Student and faculty shall be able to browse the digital menu and select items for their order.

FR.2. Student and faculty shall input order details such as item name, quantity, customization options and any special instructions.

FR.3. Student and faculty shall review their selected items and confirm the order before submitting it, with an estimated preparation time displayed.

3.1.4. Make Payment

FR.1. After confirming an order, the system shall automatically deduct the total amount from the user's integrated e-wallet.

FR.2. Student and faculty shall recharge their e-wallet by paying the amount at the cafeteria counter.

FR.3. Realtime payment confirmation notifications shall be sent to the user once the payment is successfully processed.

3.1.5. Feedback and Order Management

FR.1. Student and faculty shall be able to submit feedback or rate their orders (e.g., food quality, delivery time) from their dashboard after order completion.

FR.2. Admins shall review feedback regularly and make improvements to the service based on suggestions.

FR.3. Admins shall have access to real-time order management, monitoring orders from preparation to delivery completion.

3.1.6. View Menu

FR.1. Student and faculty shall be able to access a digital menu that is updated in real-time, displaying all available items along with their prices.

FR.2. The system shall highlight items that are out of stock in real-time to prevent orders for unavailable items.

3.1.7. Order from Office (Faculty)

FR.1. Faculty members shall be able to place meal orders directly from their office through the platform and request delivery to their specified location.

FR.2. Orders placed by faculty within a 30-minute window shall be grouped together for more efficient delivery.

3.1.7. Give Feedback

FR.1. Student and faculty shall be able to submit complaints, suggestions, and general feedback on their experience, including food quality and service.

FR.2. Feedback submission shall be integrated into the user's dashboard for easy access and submission after orders.

FR.3. Student and faculty shall be required to enter their name, email address, and a comment when submitting feedback.

3.1.8. Manage Orders (Admin)

FR.1. Admins shall have real-time access to monitor and manage all orders, from preparation to delivery, through a centralized management dashboard.

FR.2. The system shall provide real-time status updates to both student and faculty and admins on order progress.

3.1.9. Review Feedback (Admin)

FR.1. Admins shall review user feedback, analyze trends, and identify actionable insights to improve food and service quality.

3.1.10. Schedule Preorders

FR.1. Student and faculty shall have the option to schedule their orders up to one day in advance through their dashboard.

3.1.11. Manage Menu (Admin)

FR.1. Admins shall have the ability to update the menu in real-time, including marking items as out of stock or introducing new items.

FR.2. All menu changes shall be immediately reflected in the system and visible to student and faculty.

3.1.12. Deliver Group Orders (Faculty)

FR.1. Faculty orders placed within a 30minute window shall be grouped for collective delivery to ensure operational efficiency.

FR.2. Faculty student and faculty shall be notified of the estimated delivery time for group orders.

3.1.13. Cancel Order

FR.1. Student and faculty shall be able to cancel their order within a predefined timeframe (e.g., before food preparation starts).

FR.2. There will be a minimum deduction from the user's e-wallet balance upon cancellation, depending on how close to the preparation time the cancellation occurs.

FR.3. Cancellation confirmations shall be sent immediately upon successful cancellation.

3.1.14. Manage Records (Admin)

FR.1. Admins shall have access to comprehensive transaction records, user activity logs, and sales data.

3.1.15. Notifications

FR.1. The system shall send real-time notifications to student and faculty for order confirmations, payment deductions, estimated delivery times, and feedback submissions.

FR.2. A reminder system for the cafeteria admin, prompting them to publish the menu for the next day. This reminder should be sent at a configurable time (e.g., 24 hours prior to the next day's start).

3.1.16. Recharge E-wallet

FR.1. Student and faculty shall recharge their e-wallet balance manually by making payments at the cafeteria counter.

FR.2. The system shall update the user's balance in real-time after the manual recharge has been processed, and student and faculty will receive confirmation via notification.

3.1.17. Pre-orders:

FR.1. Student and faculty can place orders in advance for a specific time or date, allowing the cafeteria to manage demand and improve service efficiency.

FR.2. The system should notify the user once their pre-order is ready for pickup or delivery.

3.1.18. Manage Pre-orders:

FR.1. The system shall allow admin to receive pre-order request.

FR.2. Admin can approve or reject user request.

FR.3. Admin will be able to manage pre-order requests in real-time.

3.1.19. Bulk Orders:

FR.1. Students and faculty can place bulk orders in advance, enabling the cafeteria to better manage large-quantity demands.

FR.2. The system should notify the user once their bulk order is ready for pickup or delivery.

3.1.20. Manage Bulk Orders:

FR.1. The system shall allow the admin to receive bulk order requests.

FR.2. The admin can approve or reject bulk order requests.

FR.3. The admin shall be able to manage and track bulk order requests in real time.

3.1.21. View Map:

FR.1. The system shall provide the attendant with a map interface to view the location and delivery status of current orders.

FR.2. The attendant shall be able to view all ongoing and pending delivery orders assigned to them.

FR.3. The attendant shall be able to update the order status to “Delivered” once the order has been successfully delivered.

FR.4. The system shall reflect real-time updates of order status for students, faculty and admin.

3.1.22. View Regular Orders (Chef):

FR.1. The system shall allow the chef to view all current-day food orders placed by students and faculty.

FR.2. The chef shall be able to view order details such as item name, quantity, and user type (student or faculty).

FR.3. The chef shall receive real-time updates when new orders are placed or modified.

3.1.23. View Pre-orders (Chef):

FR.1. The system shall allow the chef to view all pre-orders placed by students and faculty for future dates.

FR.2. The chef shall be able to filter pre-orders by user type, item, or scheduled date.

FR.3. The system shall notify the chef in advance of pre-orders to facilitate timely preparation.

3.1.24. View Bulk Orders (Chef):

FR.1. The system shall allow the chef to view all bulk order requests made by students and

faculty.

FR.2. The chef shall be able to see detailed information, including quantity, user name, and any special instructions.

FR.3. The chef shall be notified of new or updated bulk orders in real time.

3.1.25. View Stock (Chef):

FR.1. The system shall allow the chef to view the available stock for the current day's food preparation.

FR.2. The system shall show ingredient quantities and low-stock alerts to help manage preparation.

FR.3. The chef shall be able to plan meals and prioritize preparation based on available stock levels.

3.1.26. Manage Stock (Admin):

FR.1. The system shall allow the admin to add stock items, including food ingredients and supplies, with quantity and unit details.

FR.2. The admin shall be able to update or delete existing stock entries at any time.

FR.3. The system shall reflect updated stock data in real time for chef visibility and food preparation planning.

FR.4. The system shall notify the chef of any critical stock updates or low-stock warnings added by the admin.

3.1.27. Track Order by Faculty:

FR.1. The system shall allow faculty members to track the status of their placed orders in real time.

FR.2. The system shall display status updates such as "Order Received," "In Preparation," "Out for Delivery," and "Delivered."

FR.3. The system shall show the estimated and actual time of each status update to help faculty monitor delivery progress.

3.1.28. Share Receipt:

FR.1. The system shall generate a digital receipt for each completed order placed by students and faculty.

FR.2. Student and faculty shall be able to download, print, and share their receipts directly from the application.

FR.3. Each receipt shall include a QR code that can be scanned to verify order details such as order ID, items, total amount, and order status.

FR.4. The QR code shall be scannable by both the cafeteria staff and system modules for quick validation.

Non-Functional Requirements

Security:

NFR-1: User data (including personal information, payment details, and order history) shall be encrypted both in transit (using HTTPS) and at rest.

NFR-2: The system shall enforce strong password requirements of at least 8 characters with a mix of uppercase, lowercase, numbers, and special characters.

Performance:

NFR-1: All page loads and API responses shall have an average response time of less than 2 seconds under normal operating conditions.

Chapter 4:

Design Document

System Design

4.1 Introduction

We will discuss system design in this chapter. This is an important step where we decide how a system will work by figuring out how its structure, parts and data will be organized. The plan is like a blueprint that helps build systems that are flexible, work well, and meet the needs of users.

4.2 Architectural Design

The architecture of a system describes its major components, their relationships (structures) and the way they interact with one another. Software requires an architectural design to represent the software design. The IEEE defines architectural design as "the process of specifying a combination of hardware and software components and their interfaces to establish a framework for the development of a computer system." In architectural design, the overall structure of the system is selected, but the internal details of major components are ignored.

4.2.1 Basic Level Architectural Design

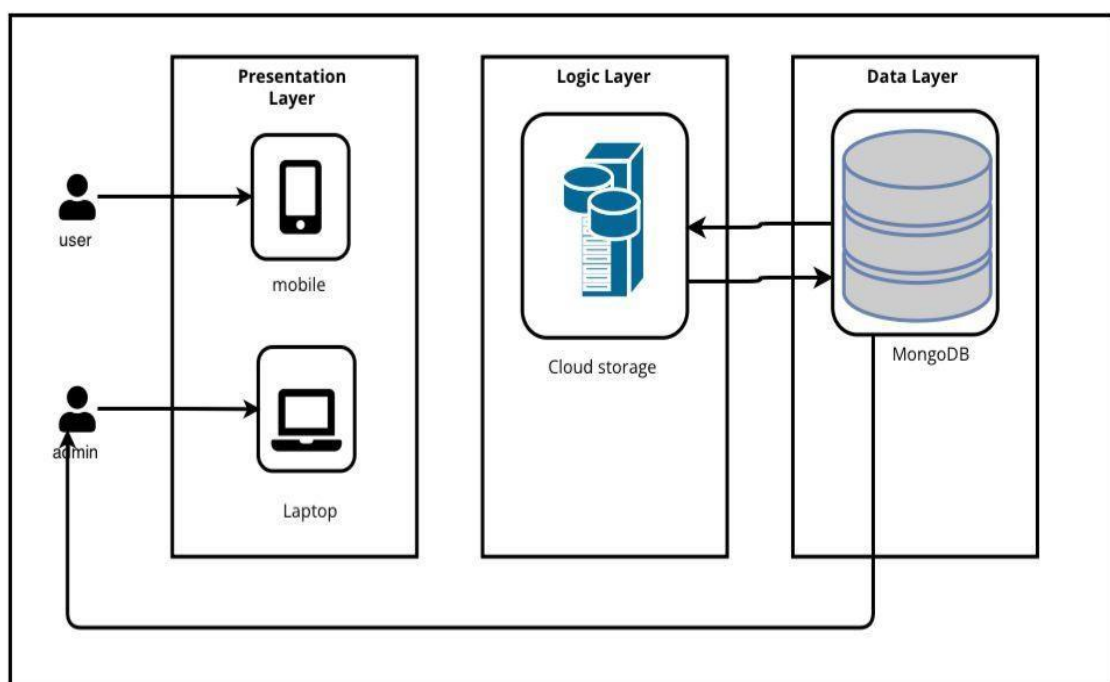


Figure 4.2.1 Basic Level Architectural Design

4.2.2 Detailed Level Architectural Design

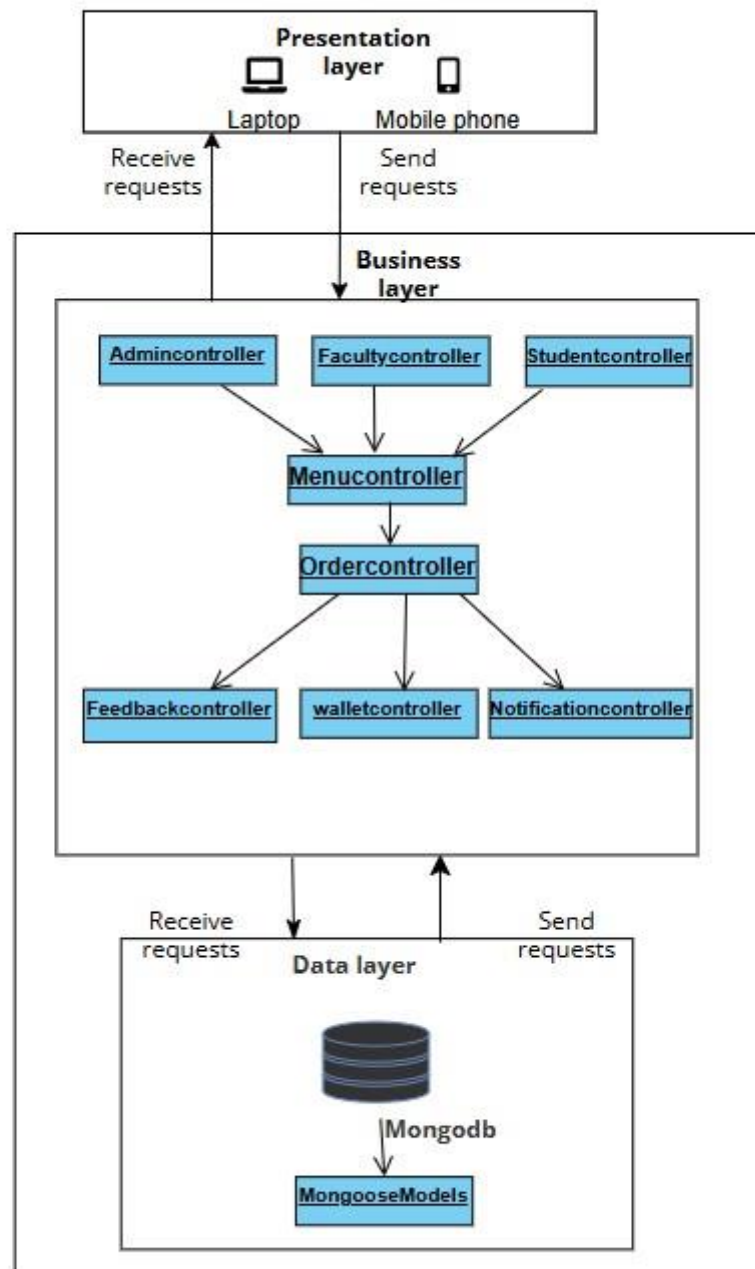


Figure 4.2.2.1 Level 1 Architectural Design Diagram

4.3 Detailed Design

Detailed design is the specification of the internal elements of all major system components, their properties, relationships, processing, and often their algorithms and data structures.

During this phase, the system architecture is broken down into smaller components, and complex details about how these components will work and interact are defined. [3]

- Structural Diagrams
- Behavioral Diagrams

4.3.1 Structural Diagrams

Structural diagrams are an important part of the UML. It represents the static aspect of the system. These diagrams help illustrate different things and objects in the system being modeled and the static parts of the diagrams are represented by classes, interfaces, objects, components and nodes. These diagrams show the things and different objects in a system being modeled.

The structural diagrams in this documentation are:

- Deployment Diagram
- Data Model Diagram
- Package Diagram
- Json Tree Diagram

4.3.2 Deployment Diagram

A deployment diagram is a type of UML diagram that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware that connects them. Deployment diagrams are commonly used to visualize the physical hardware and software of a system. Using deployment diagrams, one can gain insight into how the system will be physically implemented on the hardware.

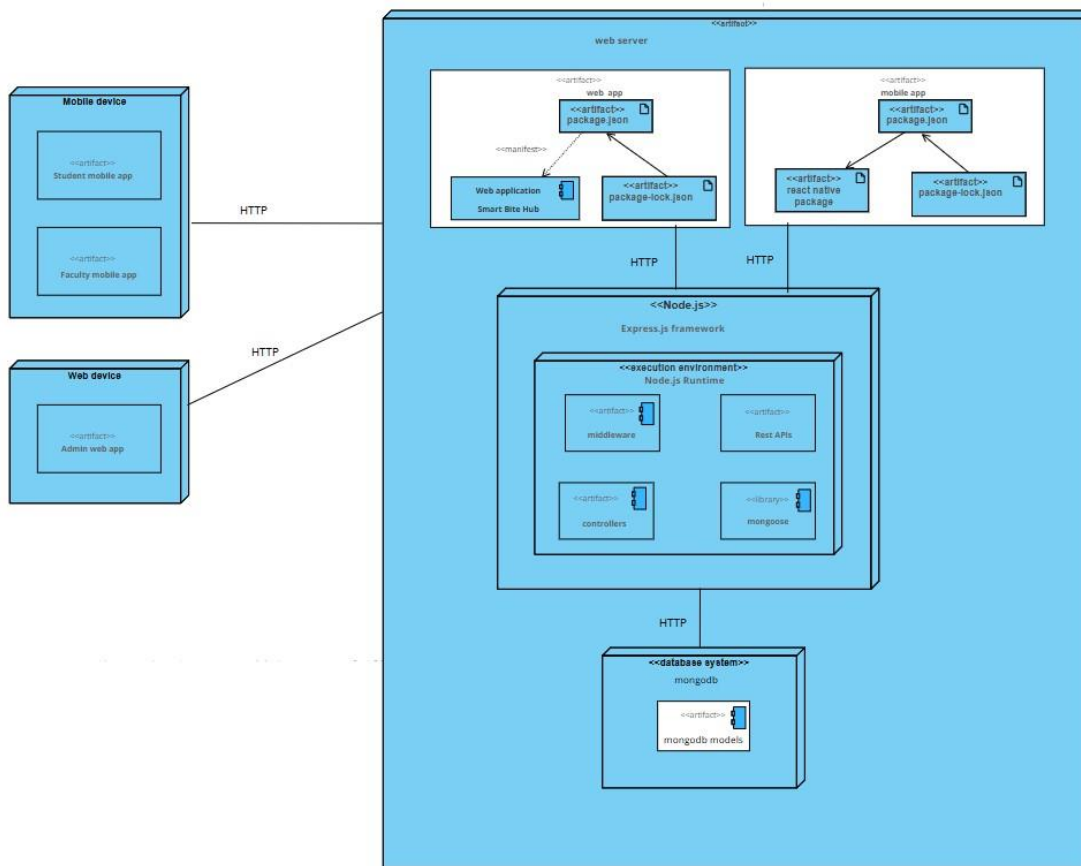


Figure 4.3.2.1 Deployment Diagram

4.3.3 JSON Tree Diagram

In software engineering, a is a type of static structure diagram that represents the hierarchical organization of data in a JSON (JavaScript Object Notation) format. It visualizes the relationships between parent and child nodes, the keys and values in objects, and the elements in arrays. Unlike class diagrams, which focus on the technical implementation of methods and classes, a JSON tree diagram provides a high-level structural view of the data, making it easier to understand how information is organized and nested.

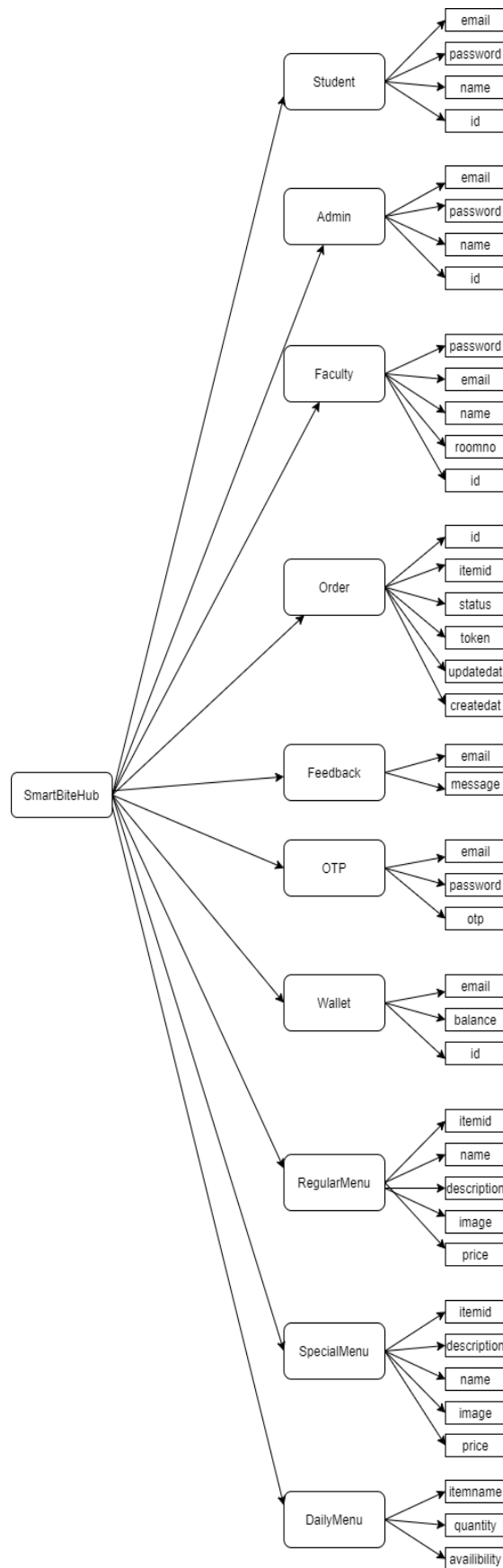


Figure 4.3.3.1 JSON Tree Diagram

4.3.4 Package Diagram

Package diagram is a kind of structural diagram that is used to organize high level systems. Packages are used for organizing large systems which contain diagrams, documents and other key deliverables. Package diagrams are often used to hide complexity. It helps us to understand the relationships and interactions between different components.

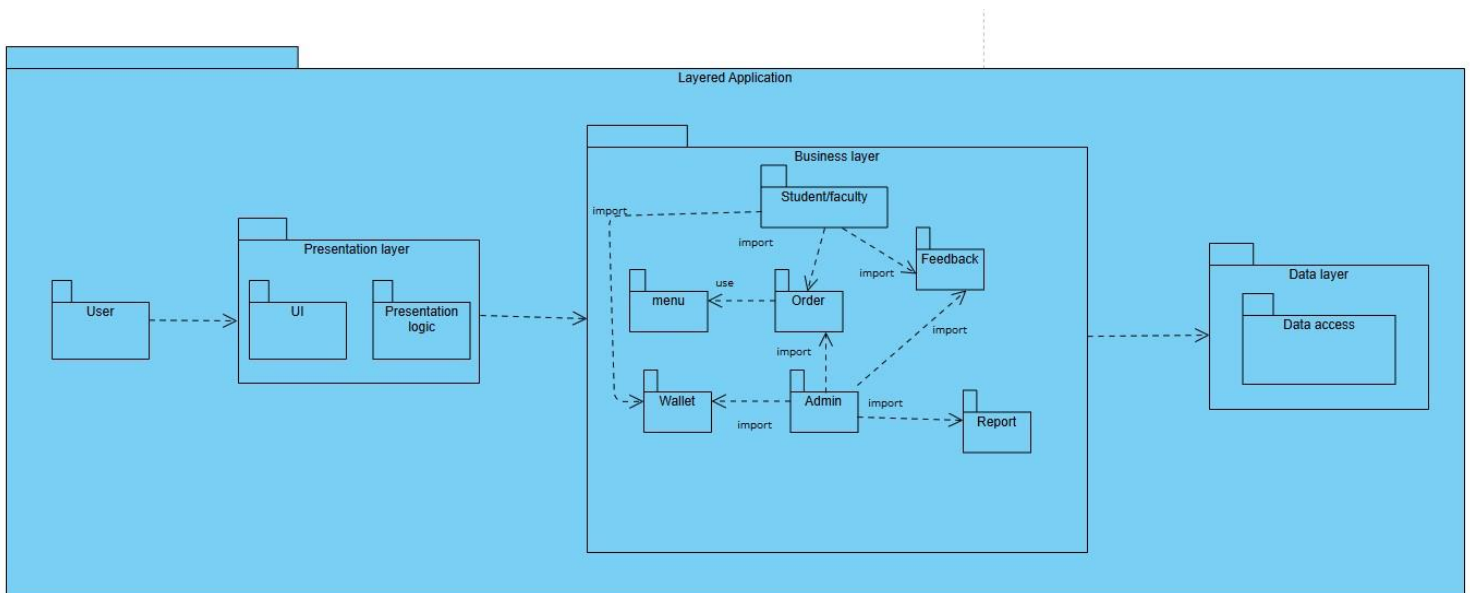


Figure 4.3.4.1 Package Diagram

4.3.2 Behavioral Diagram

UML Behavioral Diagrams show the components of a system that communicate the dynamic concepts of the system and their interrelationships. In these diagrams, the elements are comparable to verbs in a sentence, while the connections illustrate the sequence of events. We have designed use case diagrams, sequence diagrams and activity diagrams to show the dynamic interaction of a system.

4.3.2.1 Use Case Diagram

Use case diagrams are considered for high level requirement analysis of a system. It is used to gather the requirements of a system including internal and external influences. The purpose of a use case diagram is to capture the dynamic aspect of a system.

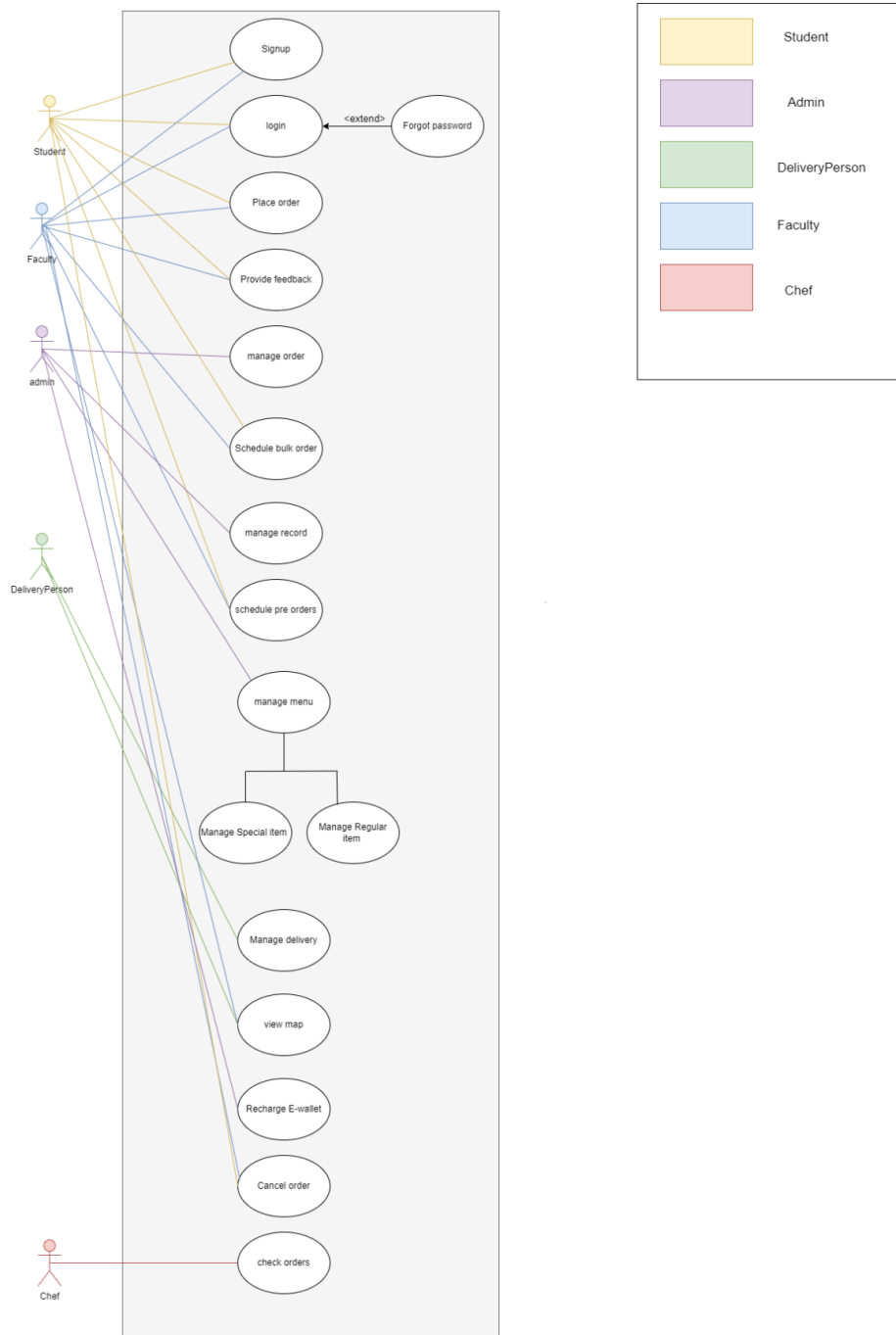


Figure 4.3.2.1 Use Case Design

4.3.2.2 System Sequence Diagram

A system sequence diagram (SSD) is a type of interaction diagram that illustrates the sequence of interactions between external actors and the system for a particular scenario or use case. SSDs focus on the system as a single entity and show the inputs and outputs exchanged between users (or other systems) and the system itself in a specific order. These diagrams are useful for capturing system requirements and understanding the flow of events, which helps in analyzing the system's behavior and designing interactions for the new or existing processes. ***For details take a look at Appendix-A.***

4.3.2.3 Activity Diagram

Activity Diagrams are used to model sequence of actions as part of the process flow. It is used to model sequence of actions to capture the process flow actions and its results. It focuses on the work performed in the implementation of an operation (a method), and the activities in a use case instance or in an object. ***For details see Appendix B***

4.4 Fully Dressed Use Cases

Use Case 1: Signup

UC-ID	UC-001
Name	Create Account
Authors	Vareesha
Priority	High
Criticality	Critical
Source	Project Documentation
Responsible	System
Description	Updated to reflect student, faculty, and admin registration processes, with room numbers for faculty.
Trigger event	User selects "Create Account".

Actors	User (Faculty, Student), Admin, Delivery Person
Postcondition	User account created with roles assigned.
Result	Account created successfully.
Main Scenario	1. User selects "Create Account". 2. System displays the registration form. 3. Students enter their email and password. 4. Faculty enter their email, password, and room number for delivery purposes. 5. Admin enters their email and password. 6. System creates the account.
Alternative Scenario	• 2a. Incorrect Email: 1. System detects the email format is wrong. 2. System prompts the user to enter the correct email address format. 3. User enters the correct email and resubmits.
Special Requirements	System must differentiate between user roles (student, faculty, admin).
Frequency of Use	Rare, as users typically create accounts only once.
Assumptions	Users are familiar with the account creation process and understand the required format for email and password.

Use Case 2: Faculty Signup

UC-ID	UC-002
Name	Create Account
Authors	Vareesha
Priority	High
Criticality	Critical
Source	Project Documentation

Responsible	System
Description	Updated to reflect faculty registration processes, with room numbers.
Trigger event	User selects "Create Account".
Actors	Faculty
Postcondition	Faculty account created.
Result	Account created successfully.
Main Scenario	1. Faculty selects "Create Account". 2. System displays the registration form. 3. Faculty enter their email, password, and room number for delivery purposes. 4. System validates the email format contains "@" and a domain and password strength contains uppercase, lowercase, numbers, and special characters. 5. System creates the account.
Alternative Scenario	3a. Incorrect Email: 1. System detects the email format is wrong. 2. System prompts the user to enter the correct email address format. 3. User enters the correct email and resubmits. 3b. Weak Password: 1. System detects the password is too weak. 2. System prompts the user to enter a stronger password. 3. User enters a stronger password and resubmits.
Special Requirements	Registration forms may have different fields depending on the selected role
Frequency of Use	Rare, as users typically create accounts only once.
Assumptions	Users are familiar with the account creation process and understand the required format for email and password.

Use Case 3: Login

UC-ID	UC-003
--------------	---------------

Name	Login
Authors	Vareesha
Priority	High
Criticality	High
Source	Project Documentation
Responsible	System
Description	Enables users to log in using their credentials.
Trigger event	User selects "Login" on the dashboard.
Actors	User (Faculty, Student, Admin)
Precondition	User has an existing account.
Postcondition	User is logged in and redirected to their dashboard.
Result	User successfully logged in.
Main Scenario	1. User selects the "Login" option on the dashboard. 2. System prompts the user to enter their email and password. 3. User enters the registered email and password. 4. System verifies the email and password. 5. If the credentials are correct, the user is logged in. 6. System redirects the user to their role-specific dashboard (faculty/student/admin).
Alternative Scenario	• 3a. Incorrect Email: 4. System detects the email does not exist. 5. System prompts the user to enter the correct email. 6. User enters the correct email and resubmits. • 4a. Incorrect Password: 1. System detects the password is incorrect. 2. System prompts the user to re-enter the correct password.

	3. User enters the correct password and proceeds.
Frequency of Use	Frequent, as users log in regularly to access their dashboard.
Assumptions	Users have a valid account and remember their credentials.

Use Case 4: Place order

UC-ID	UC-004
Name	Place Digital Order
Authors	Vareesha
Priority	High
Criticality	Critical
Source	Project Documentation
Responsible	System
Description	Allows users to place a meal order digitally.
Trigger event	User selects an item from the digital menu.
Actors	User (Student, Faculty)
Precondition	User is logged in and viewing the digital menu.
Postcondition	Order is placed and payment is deducted from the user's e-wallet.
Result	Order is successfully placed.
Main Scenario	1. User navigates to the digital menu. 2. System displays available menu items with pricing. 3. User selects one or more items from the menu. 4. User confirms their selection and proceeds to checkout. 5. System displays the order summary. 6. User confirms the order and makes the payment from their e-wallet. 7. System deducts the amount from the e-wallet and confirms the order.

	8. Order confirmation is displayed.
Alternative Scenario	• 3a. Out-of-Stock Item: 1. System detects one or more selected items are out of stock. 2. System notifies the user about the unavailable items. 3. User selects alternative available items or cancels the selection. 4. System notifies if the item is in stock again. • 6a. Insufficient Balance in E-wallet: 1. System detects that the e-wallet balance is insufficient. 2. System prompts the user to recharge their e-wallet or reduce the order amount. 3. User recharges the e-wallet or modifies the order and proceeds.
Special Requirements	System must display real-time availability of menu items and allow for immediate payment via e-wallet.
Frequency of Use	Occasional, based on user's meal ordering needs.
Assumptions	Users have sufficient e-wallet balance and understand the menu interface.

Use Case 5: Make payment

UC-ID	UC-005
Name	Make Payment
Authors	Hira
Priority	High
Criticality	Critical
Source	Project Documentation
Responsible	System
Description	Deducts payment for an order from the user's e-wallet.
Trigger event	User confirms the order.

Actors	User (Student, Faculty)
Precondition	User has selected items for an order.
Postcondition	Payment is deducted from the user's e-wallet.
Result	Payment is successfully processed.
Main Scenario	1. User confirms the order at checkout. 2. System calculates the total order amount. 3. System checks the e-wallet balance. 4. System deducts the total amount if sufficient funds are available. 5. Payment confirmation is displayed to the user. 6. Order confirmation is sent to the user.
Alternative Scenario	• 3a. Insufficient Funds: 1. System detects that the e-wallet balance is insufficient. 2. System prompts the user to recharge their e-wallet or modify the order. 3. User recharges the e-wallet or modifies the order and resubmits.
Special Requirements	Real-time e-wallet balance updates and payment processing must be reliable.
Frequency of Use	Frequent, whenever orders are placed.
Assumptions	Users have sufficient balance and payment systems are functional.

Use Case 6: Cancel order

UC-ID	UC-006
Name	Cancel Order
Authors	Hira
Priority	Medium
Criticality	Medium

Source	Project Documentation
Responsible	System
Description	Allows users to cancel their order within a specific timeframe.
Trigger event	User selects "Cancel Order".
Actors	User (Student, Faculty)
Precondition	User has placed an order.
Postcondition	Order is canceled, and a partial refund is processed.
Result	Order successfully canceled.
Main Scenario	1. User navigates to the order. 2. User selects the order they wish to cancel. 3. System checks if the order is within the cancellation window (e.g., 5 minutes after placement). 4. If eligible, system prompts the user to confirm the cancellation. 5. User confirms the cancellation. 6. System cancels the order and deducts 5% cancellation fee of the total from the user's e-wallet.
Alternative Scenario	• 3a. Order Not Eligible for Cancellation: 1. System detects that the order is outside the cancellation window. 2. System notifies the user that the order cannot be canceled. 3. User decides to keep the order or contact support for assistance.
Special Requirements	System must enforce the cancellation window and process partial refunds accurately.
Frequency of Use	Rare, as orders are generally not canceled often.
Assumptions	Users understand the cancellation policy and timeframe.

Use Case 7: Manage Menu

UC-ID	UC-007
Name	Manage Menu
Authors	Hira
Priority	Medium
Criticality	Low
Source	Project Documentation
Responsible	System
Description	Allows admin to manage the available menu items, including adding, updating, and deleting menu items. Admin can view and edit menu details such as item names, descriptions, pricing, and availability.
Trigger event	Admin selects "Manage Menu".
Actors	Admin
Precondition	Admin is logged in with registered account.
Postcondition	Menu items are managed successfully; updates are reflected in real-time for users.
Result	Menu is successfully displayed and updated.
Main Scenario	1. Admin navigates to the "Manage Menu" page. 2. System retrieves the current menu from the database. 3. Admin views the available menu items. 4. Admin selects an option to add, update, or delete a menu item. 4a. Add Menu Item: 1. Admin selects "Add Item". 2. System displays a form for new item details (name, description, price, availability). 3. Admin enters the item details and submits the form. 4. System adds the new item to the database and updates the menu.

	5. System confirms the item has been added successfully. 4c. Delete Menu Item: 1. Admin selects an item to delete. 2. System prompts for confirmation. 3. Admin confirms the deletion. 4. System removes the item from the database and updates the menu. 5. System confirms the item has been deleted successfully.
Alternative Scenario	2a. Menu Retrieval Failure: 1. System fails to retrieve the menu due to a database issue. 2. System displays an error message. 3. User tries again later.
Special Requirements	System must retrieve and display real-time menu updates from the database.
Frequency of Use	Frequent, as users check the menu regularly.
Assumptions	Menu is regularly updated and available for users.

Use Case 8: Recharge Wallet

UC-ID	UC-008
Name	Recharge E-Wallet
Authors	Alishba
Priority	High
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Allows admins to recharge user's e-wallet balance.
Trigger event	User requests to Recharge E-Wallet.

Actors	Admin
Precondition	Admin is logged in and has access to payment methods.
Postcondition	E-wallet is recharged, and the balance is updated.
Result	Successful recharge of e-wallet.
Main Scenario	1. Admin receives the cash and updates the e-wallet balance in the system. 2. System processes the update and adds the specified amount to the user's e-wallet balance. 3. User receives a confirmation notification showing the updated balance.
Alternative Scenario	• 2a. Payment Failure: 1. System encounters an error during updating balance processing. 2. System displays a payment failure message. 3. Admin retries the wallet update.
Special Requirements	Payment processing must be secure and reliable. System should support multiple payment methods.
Frequency of Use	Occasional, depending on the user's spending needs.
Assumptions	Users have valid payment methods and know how to use the recharge feature.

Use Case 10: Manage order

UC-ID	UC-010
Name	Manage Orders
Authors	Vareesha
Priority	High
Criticality	High
Source	Project Documentation
Responsible	Admin

Description	Admin can monitor and manage orders in real-time.
Trigger event	Admin accesses the "Manage Orders" dashboard.
Actors	Admin
Precondition	Orders are being placed by users.
Postcondition	Orders are successfully managed.
Result	Admin monitors and manages all active orders.
Main Scenario	1. Admin navigates to the "Manage Orders" dashboard. 2. System displays a list of all active orders. 3. Admin reviews the order details (e.g., user, items, payment status). 4. Admin can update the order status (e.g., processing, delivered). 5. Admin monitors the real-time order status updates. 6. Admin identifies any delayed or problematic orders and takes corrective action.
Alternative Scenario	• 3a. System Error in Order Details: 1. Admin detects an issue with order details (e.g., missing items). 2. Admin contacts the user or corrects the issue manually.
Special Requirements	Admin must have real-time visibility of orders, statuses, and corrective options.
Frequency of Use	Frequent, as orders are constantly being placed and processed.
Assumptions	Admin is familiar with the system interface for managing orders.

Use Case 11: Give feedback

UC-ID	UC-011
Name	Give Feedback
Authors	Hira
Priority	Medium

Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Users can submit feedback regarding their orders.
Trigger event	User selects "Give Feedback".
Actors	User (Faculty, Student)
Precondition	User has completed an order.
Postcondition	Feedback is submitted successfully.
Result	Feedback is recorded and available for admin review.
Main Scenario	1. User selects the "Give Feedback" option for a specific order. 2. System displays a feedback form. 3. User provides name, email and comments. 4. User submits the feedback. 5. System confirms that the feedback has been recorded.
Alternative Scenario	• 3a. Incomplete Feedback Form: 1. System detects that mandatory fields are incomplete. 2. System prompts the user to fill in the missing details. 3. User completes the form and resubmits.
Special Requirements	Feedback must be linked to specific orders for detailed analysis.
Frequency of Use	Occasional, depending on user satisfaction or issues with their orders.
Assumptions	Users provide honest feedback to help improve the system.

Use Case 12: Review Feedback

UC-ID	UC-012
Name	Review Feedback

Authors	Hira
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	Admin
Description	Admin can review feedback submitted by users.
Trigger event	Admin accesses the "Feedback" dashboard.
Actors	Admin
Precondition	Users have submitted feedback.
Postcondition	Feedback is reviewed and addressed.
Result	Feedback successfully reviewed.
Main Scenario	1. Admin navigates to the "Feedback" dashboard. 2. System displays user-submitted feedback. 3. Admin reviews feedback details (e.g., rating, comments). 4. Admin identifies areas for improvement based on feedback. 5. Admin takes corrective actions (if necessary). 6. Admin marks the feedback as "Reviewed".
Alternative Scenario	• 3a. No Action Required: 1. Admin reviews feedback that requires no further action. 2. Admin marks the feedback as "Resolved".
Special Requirements	Feedback must be easily accessible to admin with relevant details for action.
Frequency of Use	Occasional, based on the frequency of user feedback submissions.
Assumptions	Admin will act on feedback promptly to improve user experience.

Use Case 13: Schedule Pre-order

UC-ID	UC-013
Name	Schedule Pre-orders
Authors	Vareesha
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Users can schedule orders up to a day in advance.
Trigger event	User selects "Schedule Pre-order".
Actors	User (Faculty, Student)
Precondition	User is logged in and has access to the menu.
Postcondition	Pre-order is scheduled for future processing.
Result	Order successfully scheduled.
Main Scenario	1. User navigates to the "Pre-order". 2. System displays available items and scheduling options. 3. User selects items for the pre-order and schedules the time (up to 24 hours in advance). 4. System confirms the scheduled order. 5. System processes the payment from the e-wallet. 6. System schedules the order for future preparation and delivery.
Alternative Scenario	• 3a. Item Not Available for Pre-order: 1. System detects that one or more items are unavailable for pre-order. 2. System notifies the user and suggests alternative items. 3. User modifies the pre-order or cancels it.

Special Requirements	System must allow flexible scheduling and provide clear availability for future orders.
Frequency of Use	Occasional, based on user preferences for pre-ordering meals.
Assumptions	Users are aware of the pre-order feature and its benefits.

Use Case 14: Manage Records

UC-ID	UC-014
Name	Manage Records
Authors	Vareesha
Priority	High
Criticality	Medium
Source	Project Documentation
Responsible	Admin
Description	Admin can access transaction records, user activity logs, and sales data.
Trigger event	Admin selects "Manage Records" from the dashboard.
Actors	Admin
Precondition	Transactions, user activities, and sales data exist in the system.
Postcondition	Admin has successfully viewed or exported records.
Result	Records are accessed and managed.
Main Scenario	1. Admin logs in and navigates to the "Manage Records" dashboard. 2. System displays options for viewing transaction records, user activities, and sales data. 3. Admin selects the specific data set they want to view. 4. System retrieves the requested data from the database. 5. Admin views the records and can filter, sort, or export them. 6. Admin exports the records if needed.

	7. System confirms the successful export (if performed).
Alternative Scenario	• 4a. Data Retrieval Error: 1. System fails to retrieve the records due to a database issue. 2. System displays an error message. 3. Admin retries or contacts technical support.
Special Requirements	Admin must have a clear view of records, with the ability to export or filter as needed.
Frequency of Use	Frequent, as record management is a key administrative task.
Assumptions	Data is consistently stored and retrievable by the system without issues.

Use Case 15: Send Notification

UC-ID	UC-015
Name	Send Notifications
Authors	Alishba
Priority	Medium
Criticality	Low
Source	Project Documentation
Responsible	System
Description	System automatically sends notifications to users (e.g., order confirmations, out-of-stock alerts).
Trigger event	A system event occurs (e.g., order placed, item out of stock).
Actors	System, User (Faculty, Student), Admin
Precondition	A trigger event must occur (e.g., user action, system condition).
Postcondition	Notification is sent to the user/admin.
Result	User or admin is notified of the event.

Main Scenario	1. A trigger event occurs (e.g., user places an order, item goes out of stock). 2. System identifies the event and generates a notification. 3. System sends the notification to the relevant user or admin. 4. User/admin receives the notification.
Alternative Scenario	• 4a. Notification Delivery Failure: 1. System fails to deliver the notification due to network or email server issues. 2. System logs the failure and attempts to resend. 3. User/admin receives the notification upon successful resend.
Special Requirements	Notifications must be delivered promptly and reliably, regardless of network or server load.
Frequency of Use	Frequent, as notifications are regularly triggered by system events.
Assumptions	Users rely on notifications for timely updates, and notifications are configured correctly.

Use Case 16: Cancel pre-order

UC-ID	UC-016
Name	Cancel Scheduled Pre-order
Authors	Alishba
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Users can cancel their scheduled pre-orders before the order processing begins.
Trigger event	User selects "Cancel Pre-order".

Actors	User (Faculty, Student)
Precondition	A pre-order must be scheduled, and it must still be within the cancellation window.
Postcondition	Pre-order is canceled, and the payment (if any) is refunded.
Result	Pre-order successfully canceled.
Main Scenario	1. User navigates to the pre-order already done. 2. User selects the pre-order they want to cancel. 3. System checks if the pre-order is eligible for cancellation (i.e., before processing starts). 4. System prompts the user to confirm the cancellation. 5. User confirms the cancellation. 6. System cancels the pre-order and initiates a refund if applicable. 7. User receives a confirmation of the cancellation.
Alternative Scenario	• 3a. Ineligible for Cancellation: 1. System detects that the pre-order is outside the cancellation window. 2. System notifies the user that the pre-order cannot be canceled. 3. User decides to keep the pre-order or contacts support for further assistance.
Special Requirements	The system must ensure cancellations occur only if the order hasn't been processed, and refunds are issued promptly.
Frequency of Use	Occasional, depending on user needs to cancel pre-orders.
Assumptions	Users understand the pre-order cancellation window and process.

Use Case 17: Place bulk order

UC-ID	UC-017
Name	Place bulk orders
Authors	Hira
Priority	Medium

Criticality	Medium
Source	Project Documentation
Responsible	Admin
Description	Admin can approve or reject user-submitted pre-orders for special items.
Trigger event	Admin selects "Manage Pre-orders" from the dashboard.
Actors	Admin
Precondition	Users have submitted pre-orders.
Postcondition	Pre-orders are either approved or rejected, and users are notified accordingly.
Result	Pre-orders are successfully managed.
Main Scenario	1. Admin navigates to the "Manage Pre-orders" dashboard. 2. System displays a list of pending pre-orders. 3. Admin reviews the pre-order details (e.g., items, quantity, scheduling). 4. Admin decides to approve or reject each pre-order. 5. Admin submits the decision. 6. System updates the pre-order status to approved or rejected. 7. Users are notified of the decision.
Alternative Scenario	• 4a. Pre-order Already Processed: 1. Admin detects that a pre-order has already been processed. 2. Admin cannot reject the pre-order at this stage. 3. Admin notifies the user that the pre-order has been processed.
Special Requirements	The system must handle large quantities and process bulk orders without performance issues.
Frequency of Use	Occasional, used during events or bulk ordering needs.
Assumptions	Users know the items and quantities needed for bulk orders and inventory is available.

Use Case 18: Manage Receipt

UC-ID	UC-018
Name	Manage Receipt
Authors	Hira
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Allows users to confirm their order and view a receipt. Users can download or share the receipt after their food order is confirmed.
Trigger event	User confirms order.
Actors	User (Student, Faculty)
Precondition	User has placed an order.
Postcondition	Order is confirmed and a receipt is generated.
Result	Receipt is successfully displayed, and user has options to download or share it.
Main Scenario	1. User navigates to the order confirmation page. 2. System displays the order summary and prompts the user to confirm the order. 3. User confirms the order. 4. System generates a receipt with order details (e.g., item name, quantity, price, total, etc.). 5. System displays the receipt to the user. 6. User selects an option to either download or share the receipt.
Alternative Scenario	2a. Order Not Confirmed: 1. User decides not to confirm the order.

	2. System retains the order in the pending state. 3. User can return to the order later to confirm or cancel it.
Special Requirements	System must update the receipt details in real-time to reflect the confirmed order.
Frequency of Use	Frequent, as users regularly confirm orders and may need to download or share receipts.
Assumptions	Users understand the process of confirming orders and the available options for managing their receipts.

Use Case 18: Manage daily menu

UC-ID	UC-018
Name	Manage Daily Menu
Authors	Alishba
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Allows the admin to manage the daily menu by adding items (regular or special), specifying the quantity, product name, and availability. The admin can see the list of added items and delete items from the list if needed.
Trigger event	Admin selects "Manage Daily Menu".
Actors	Admin
Precondition	Admin is logged in with appropriate permissions.
Postcondition	Daily menu is updated successfully with the changes reflected in real-time.
Result	Daily menu is managed successfully and displayed.
Main Scenario	1. Admin navigates to the "Manage Daily Menu" page.

	- System displays options to add new menu items. - Admin selects whether the item is "Regular" or "Special". - Admin enters the product name, quantity, and availability status for the menu item. - Admin submits the details. - System adds the new item to the daily menu and displays the updated list. - Admin can see the list of added items along with their details (product name, quantity, availability). - Admin selects an item to delete. - System prompts for confirmation of deletion. - Admin confirms the deletion. - System removes the item from the list and updates the menu.
Alternative Scenario	2a. Failed to Add Item: - System encounters an error while adding the item. - System displays an error message. - Admin retries or contacts support. 8a. Failed to Delete Item: - System encounters an error while deleting the item. - System displays an error message. - Admin retries or contacts support.
Special Requirements	System must update the daily menu in real-time to reflect any changes made by the admin.
Frequency of Use	Frequent, as the daily menu is managed regularly by admins.
Assumptions	Admins are familiar with the menu management process and understand the required format for item details.

Use Case 19: Deliver order

UC-ID	UC-019
Name	Deliver Order

Authors	Alishba
Priority	Medium
Criticality	Medium
Source	Project Documentation
Responsible	System
Description	Allows delivery person to deliver food orders to faculty members within the campus.
Trigger event	A new order is marked as "Ready for Delivery" in the system.
Actors	Delivery Person
Precondition	1. Delivery person is logged in with appropriate permissions. 2. The order must be confirmed and prepared.
Postcondition	Order is marked as "Delivered" in the system.
Result	Faculty receives the ordered food successfully, and the system updates the delivery status.
Main Scenario	1. Delivery person logs into the app. 2. Delivery person views list of ready orders. 3. Delivery person selects an order to deliver. 4. Delivery person views map and timer for efficient route. 5. Delivery person picks up the food from the kitchen. 6. Delivery person delivers it to the faculty location. 7. Delivery person marks the order as delivered in the system.
Alternative Scenario	6a. Failed to deliver: 1. If the faculty is unavailable, the delivery person can mark the delivery as "Failed" with a reason (e.g., "Not Available") and return the order.

Special Requirements	System must update the delivery status, delivery time in real-time.
Frequency of Use	Frequent, based on order volume.
Assumptions	The system assigns deliveries efficiently. Faculty locations are accurate in the system.

Chapter 5:

Implementation

5.1 Endeavour (Team + Work + Way of Working)

5.1.1 Team

- Hira Ejaz
- Alishba
- Vareesha Naeem

5.1.2 Work

- Planning
- Requirement Gathering
- Designing
- Implementation
- Testing

5.1.3 Way of working

In every part of the Software Development Life Cycle or SDLC, there are different things we do to make sure the project goes well. We're going to describe what we do in each part of the process and how everyone in the team works together to make it happen.

5.1.3.1 Planning

Planning	Implemented by
Develop Plan	Alishba
Design Plan	Hira Ejaz
Review Plan	Vareesha Naeem

Table 2.2 Planning

5.1.3.2 Requirements Gathering

Requirements Gathering	Implemented by
Elicit and document requirements	Hira Ejaz
Literature review	Vareesha Naeem
Market survey	Alishba

Table 3.3 Requirement Gathering

5.1.3.3 Designing

Designing	Designed by
Basic level Architecture, Deployment Diagram, Activity diagram	Alishba
Package Diagram, Activity diagram	Hira Ejaz
Usecase Diagram, Sequence diagram, Activity diagram	Vareesha Naeem

Table 4.4 Designing

5.1.3.4 Implementation

Implementation	Implemented by
Student Panel	Alishba
Faculty Panel	Vareesha Naeem
Admin Panel	Hira Ejaz

Table 5.5 Implementation

5.1.3.5 Testing

Testing	Implemented by
Build Scenarios, Test cases	Alishba
Build Scenarios, Test cases	Vareesha Naeem
Build Scenarios, Test cases	Hira Ejaz

Table 6.6 Testing

5.2 Flow Control/Pseudo codes

A flowchart is a visual representation of a series of steps, presenting them in a sequential order. It is widely used in presenting the flow of algorithms, workflow or processes. Typically, a flowchart shows the steps as various types of boxes, and their sequence is indicated by connecting them with arrows.

5.2.1 Student

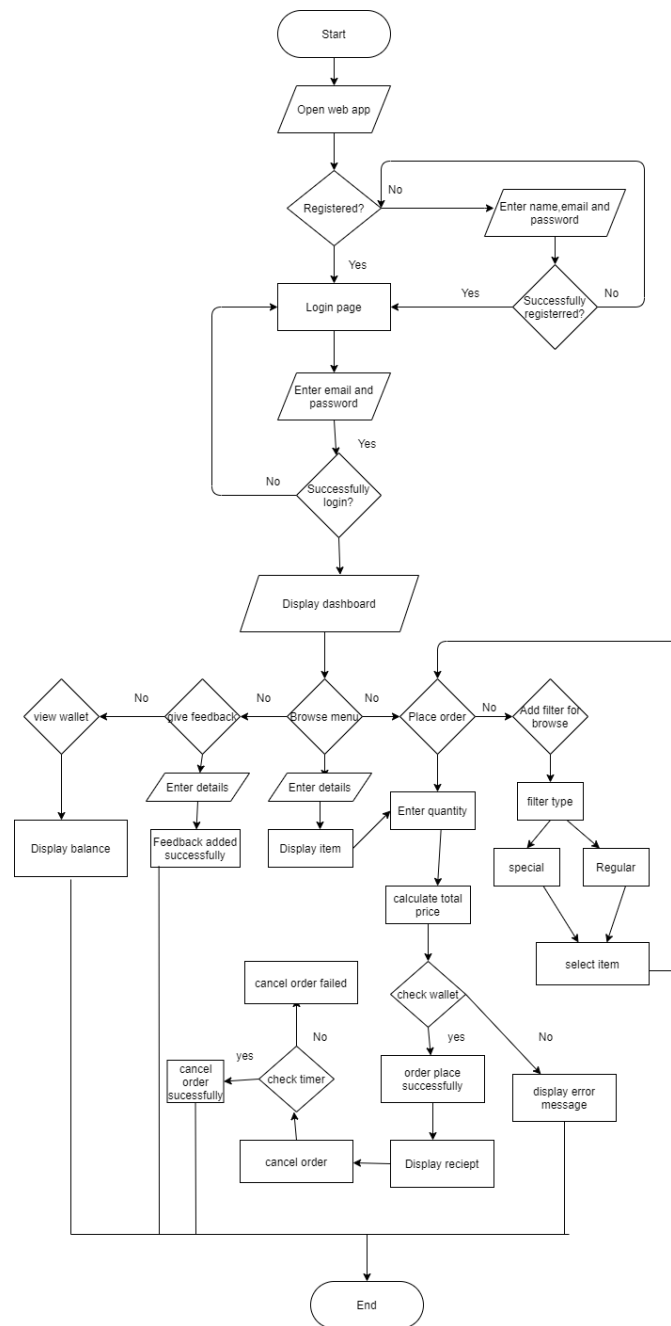


Figure 5.1 Student Flow Control

5.2.2 Faculty

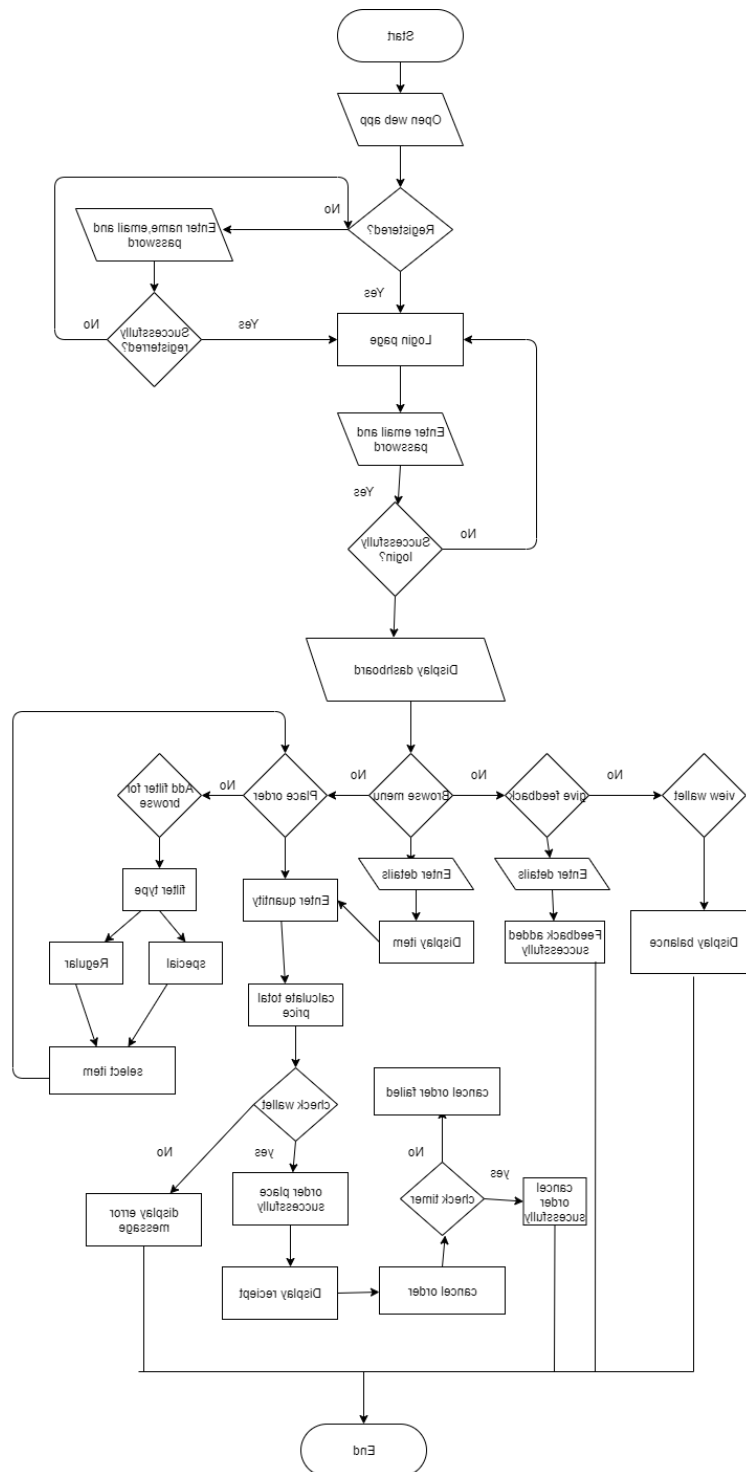


Figure 5.2 Faculty Flow Control

5.2.3 Admin

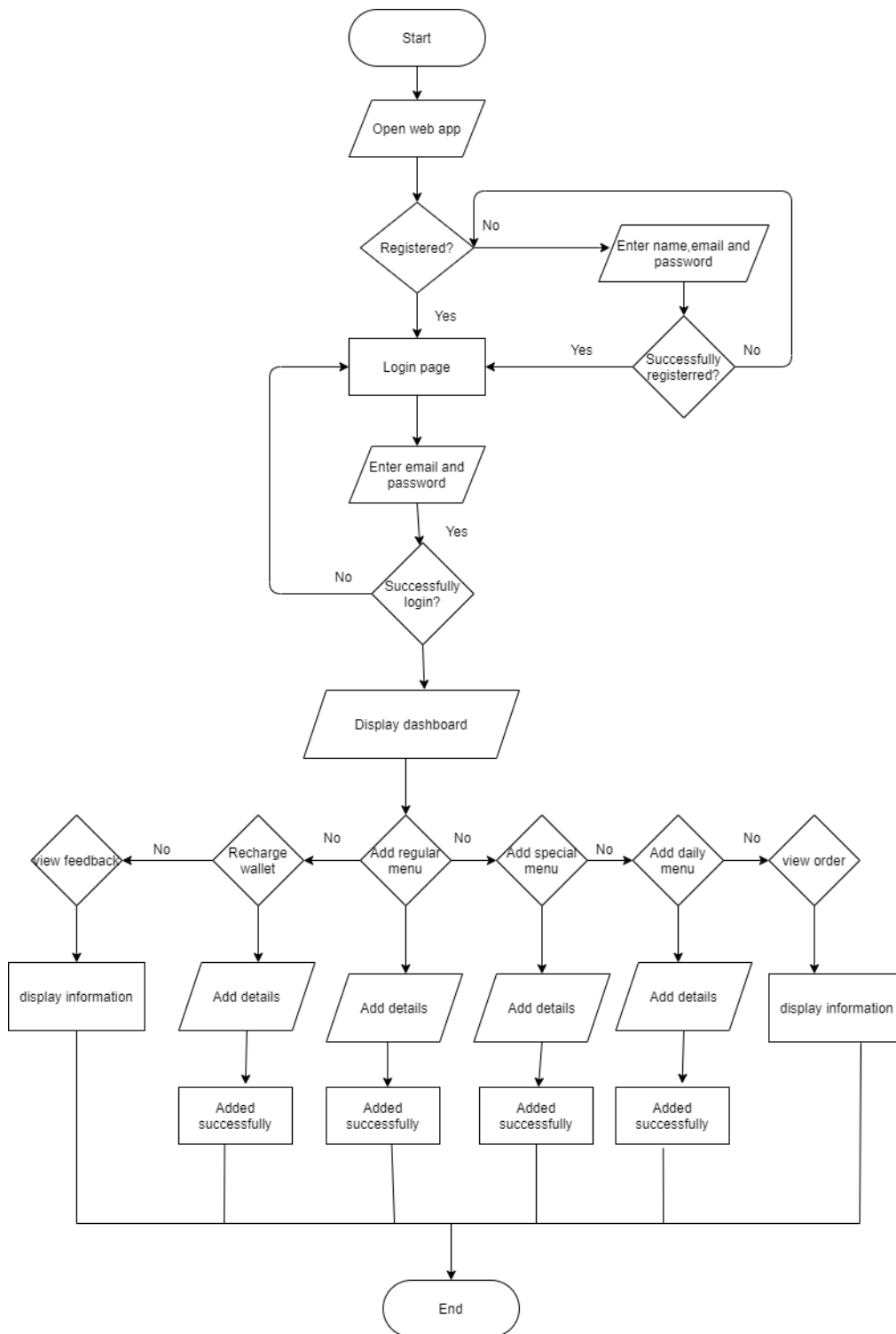


Figure 5.3 Admin Flow Control

5.2.4 Chef

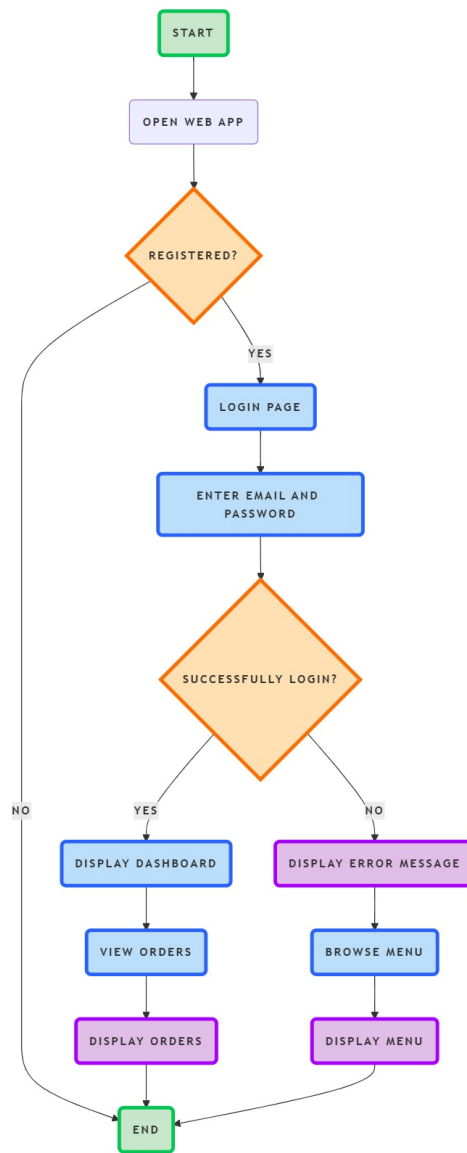


Figure 5.4 Chef Flow Control

5.2.5 Attendant

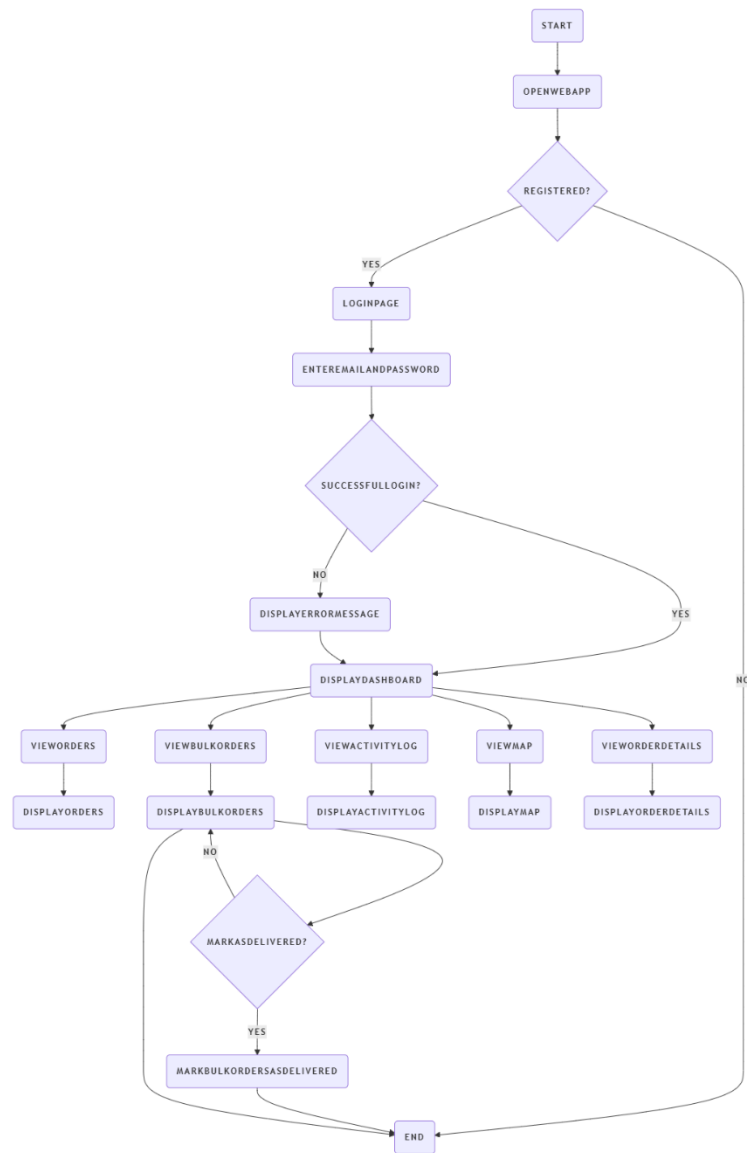


Figure 5.5 Attendant Flow Control

5.3 Components, Libraries, Web Services, and Stubs

Components and libraries are sets of pre-made, tested, and well-documented UI components that can be easily reused by integrating them across the user interface of a system.

5.1.1 Frameworks

- React Js

- React Router
- React Native
- Express
- Node Js
- Expo

5.1.2 Web Libraries/ API's Components

- Font awesome
- React
- React Icons
- Rest API
- Cors
- Dotenv
- Multer
- Axios
- Mongoose
- React Navigation/Stack
- React CLI
- Socket

5.1.3 External Libraries/ API's / Services

- Non-relational database

5.1.4 Languages

- JavaScript XML (JSX)
- MERN Stack
- React Native
- React Js
- Bootstrap
- Html
- CSS

5.1.5 Platforms/ Tools

To develop the system, following tools are used by the team members:

- Visual Studio Code
- Microsoft Teams
- GitHub
- Microsoft Word
- Microsoft PowerPoint
- Google Form
- Draw.io
- MongoDBCompass

5.2 Best Practices / Coding Standards

5.2.1 React:

- Organize components logically
- Prioritize functional components
- Utilize hooks like useState and useEffect

- Use consistent and meaningful names
- Design for different devices
- Implement responsive components

5.2.2 Node.js and Express:

- Modular code structure (controller, routes, server)
- Asynchronous programming
- Middleware separation
- Environment variables in dotenv file
- Error Handling

5.2.3 MongoDB (database):

- Schema Design Using mongoose
- Handle Database Connection

5.3 Deployment Environment

The "Smart Bite Hub" is a digital platform designed to automate the operations of the university cafeteria, addressing inefficiencies such as long queues, longer waiting times, waste of resources, unavailability of food items, and lack of record management. This innovative system enhances the overall experience for all users by making the process more convenient and efficient. An active internet connection is necessary to use this system. The cafeteria administration interacts with the Smart Bite Hub through a web application, where they can manage user registrations, track orders in real-time, update menus, and analyze feedback to continuously improve services. Meanwhile, students and faculty members use a mobile app to view the digital menu, place orders, and provide feedback. The flow of data within the system begins on the client side, where users (students and faculty) interact with the mobile app interface. The information is then sent to the server, processed, and stored in a database. The cafeteria administration accesses this data through the web app, managing and optimizing the cafeteria operations based on real-time data and user feedback.

5.3.1 Hardware Requirements:

System	Requirement
Mobile	Android, iPhone
Laptop / Desktop	Windows, Mac
RAM	4 GB
Hard Drive	10 GB
Browser	Chrome, Safari, Microsoft Edge, Firefox

Table 7.7 Hardware Requirements

5.1.1 Software Requirements:

Software	Requirement
Integrated Development Environment	Visual Studio Code
Application Programming Interface	Rest API, React API, Gmail
Programming languages	React Native, React Js and MERN Stack
Database	No SQL (MongoDB)
Package Manager	NPM, Create React App, React Router, Material-UI, Create React Native App
Version Control System	Git, Github

Table 8.8 Software Requirements

5.4 Summary

In Chapter 5, we have described how the team works together and breaks down the tasks during the different stages of creating the software. The team, which includes Alishba, Hira Ejaz, and Vareesha Naeem, does things like planning, gathering requirements, designing, implementation, and testing. Each person in the team has specific jobs to do in each of these steps. The chapter ends by talking about where and how the software will be used, including the devices and programs needed to make it work. Overall, it gives a good picture of how the team works together to make the "Smart Bite Hub " web and app.

Chapter 6:

Testing and Evaluation

6.1 Introduction

Scenario based black box testing technique is used for testing core system functionality. All scenarios are generated from the system use case diagram.

6.2 List of test scenarios

The lists of test scenarios considered for the project are:

See Appendix C: Testing

6.3 Performance and evaluation

The test cases are formulated to detect any anomalies in the customary or anticipated progression of the system. The test cases are validated, and the factual outcome corresponds to the projected outcome of the system.

Chapter 7:

Conclusion and Outlook

7.1 Introduction

In this chapter, we aim to highlight the accomplishments and insights gained during the development of the Smart Bite Hub system. Our journey included detailed research,

7.2 Critical Review

Although the system works as expected in handling core functionalities, some areas such as delivery tracking and performance optimization need further attention. More real-time updates and better admin controls could enhance usability.

7.3 Future Recommendations/Outlook

We are continuing development and plan to add features as needed. Currently, we don't have access to paid APIs, but in the future, we aim to integrate them to improve features like dynamic maps and real-time order updates, making the system more efficient and responsive.

7.4 Summary

Smart Bite Hub simplifies cafeteria services by allowing users to order food digitally and manage their orders conveniently. It improves the experience for both students and faculty, making the campus food delivery process faster and more organized. gathering user requirements, and exploring various solutions to deliver a functional and user-friendly campus food ordering and delivery system.

7.5 Achievements and Improvements

With the grace of the Almighty, we successfully implemented key features like student ordering, faculty-specific delivery, e-wallet balance deduction, and order management. This project helped us improve our technical skills, teamwork, and ability to overcome unexpected challenges collaboratively.

Reference and Bibliography

- [1] JavaPoint, “Architecture Diagram in software Engineering”, updated 2024
<https://www.javatpoint.com/architectural-design-in-software-engineering>
- [2] akshay, “Software Requirement Specification” ,2024
<https://www.geeksforgeeks.org/software-requirement-specification-srs-format/>
- [3] LucidChart, “What is System sequence Diagram”, 2024
<https://www.lucidchart.com/pages/uml-system-sequence-diagram>
- [4] GeekforGeek, “Functional vs Non-Functional Requirements”, 2024
<https://www.geeksforgeeks.org/functional-vs-non-functional-requirements/>

Appendix A: System Sequence Diagram

1. Signup

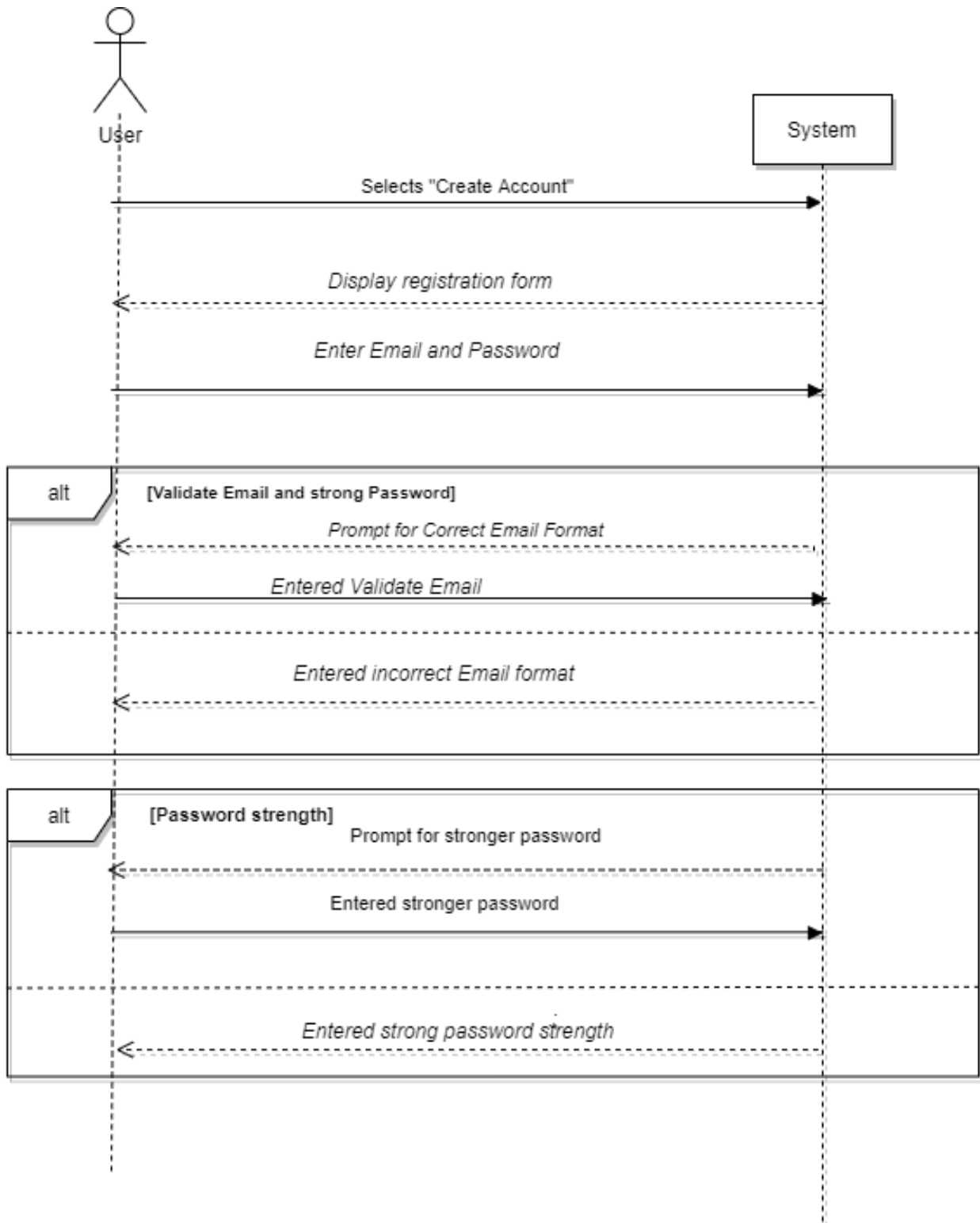


Figure A.1 Signup

2. Login

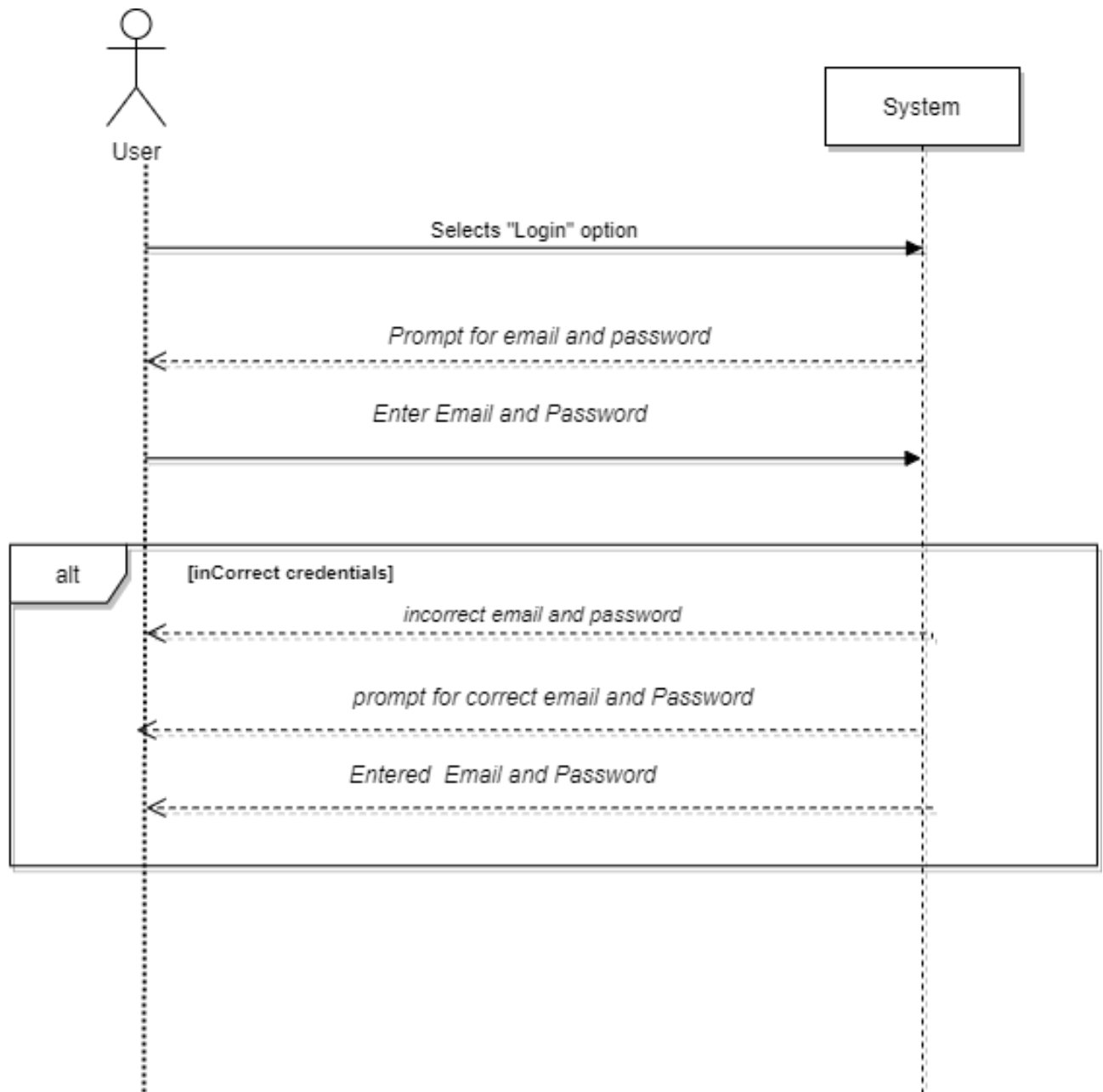


Figure A.2 Login

3. Place order

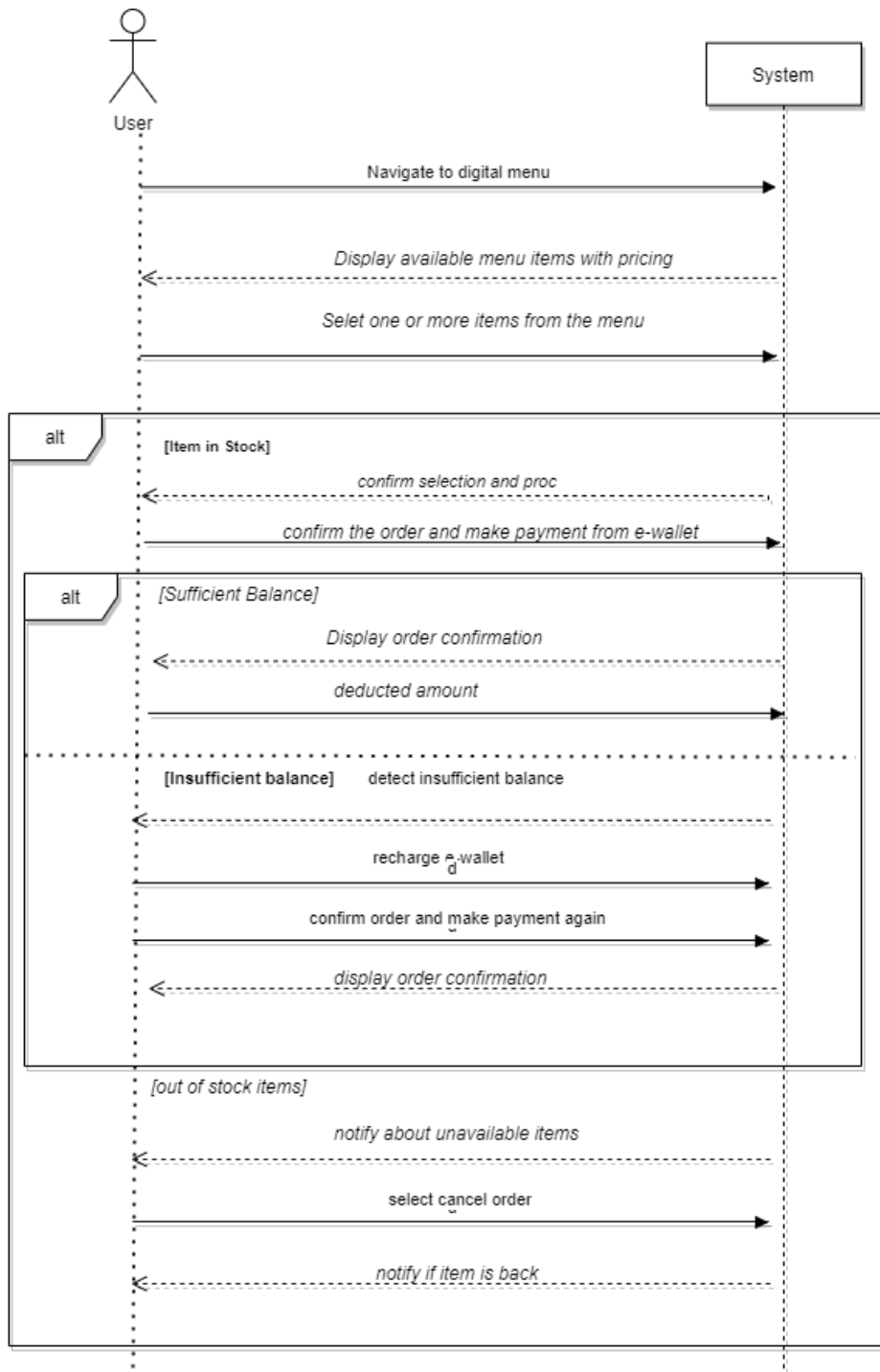


Figure A.3 Place order

4. Provide Feedback

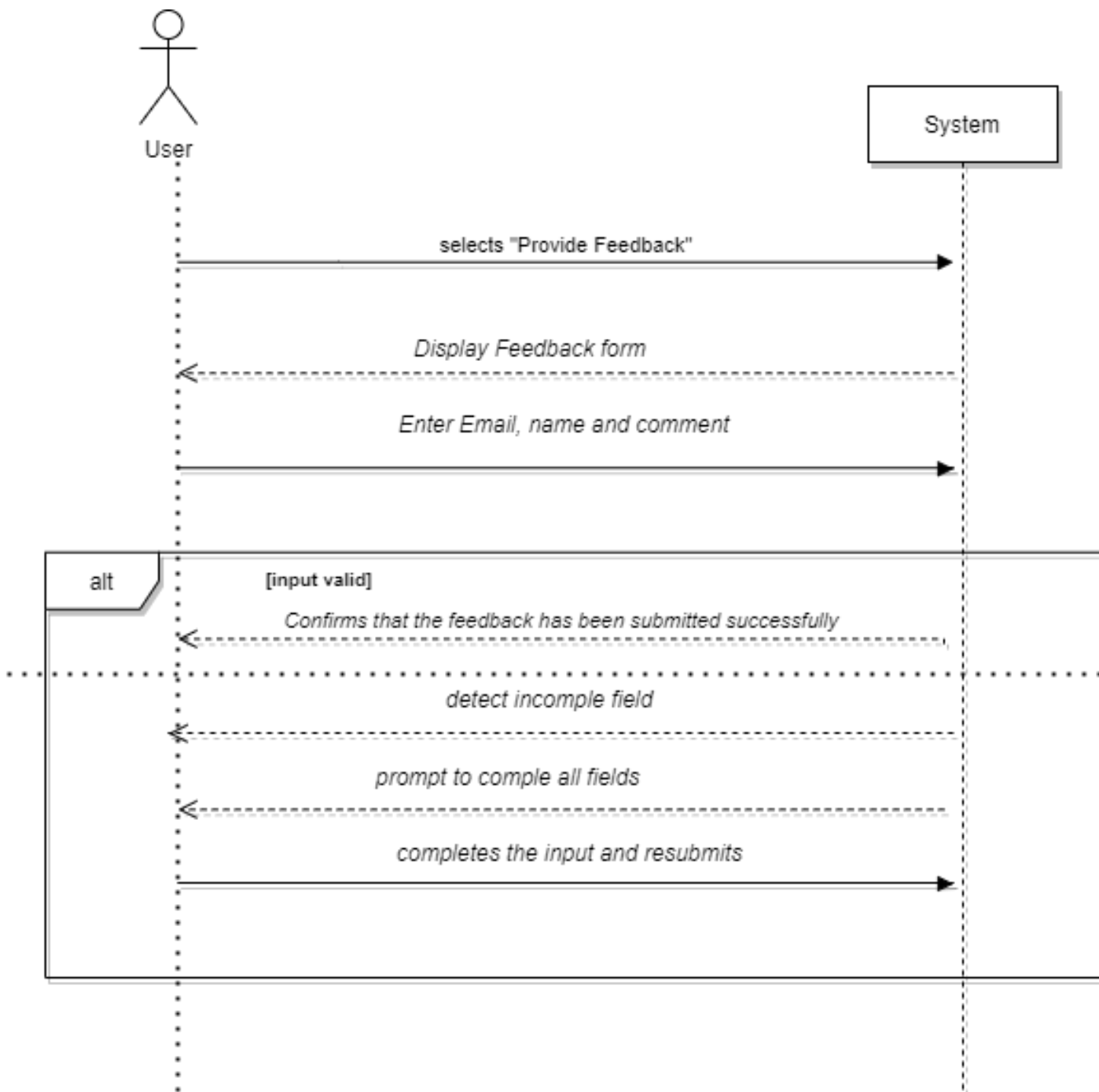


Figure A.4 Provide feedback

5. Forgot password

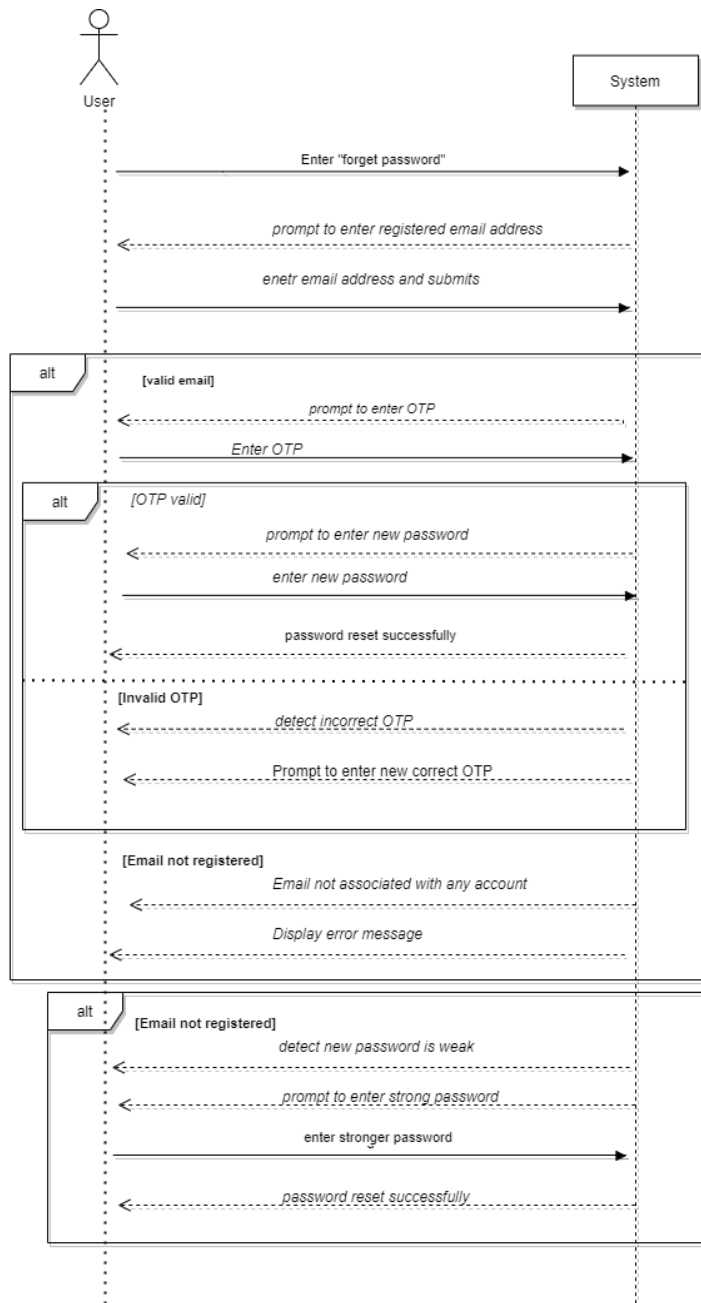


Figure A.5 Review feedback

6. Recharge wallet

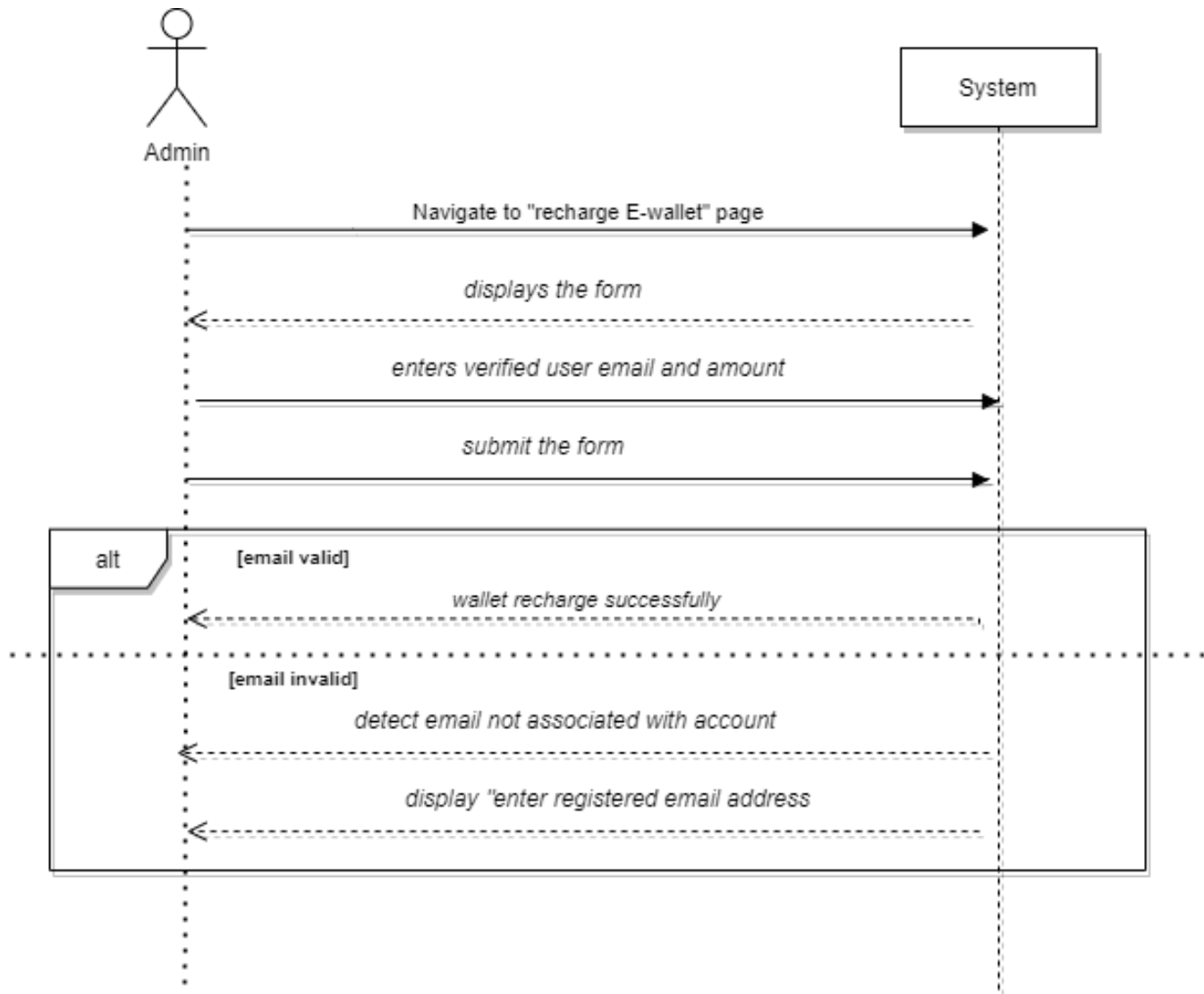


Figure A.6 Recharge wallet

7. Review feedback

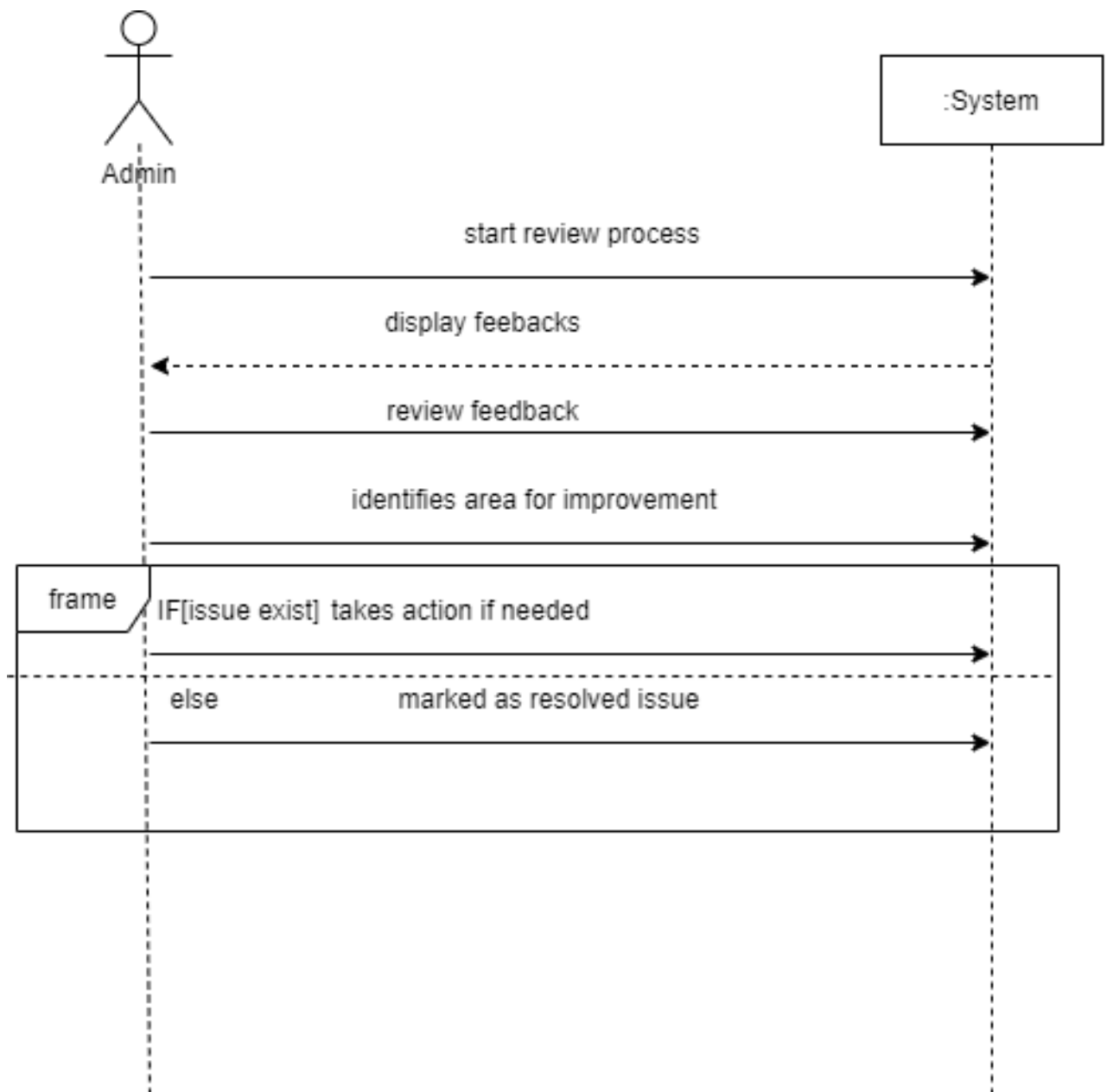


Figure A.7 Review feedback

8. Manage order

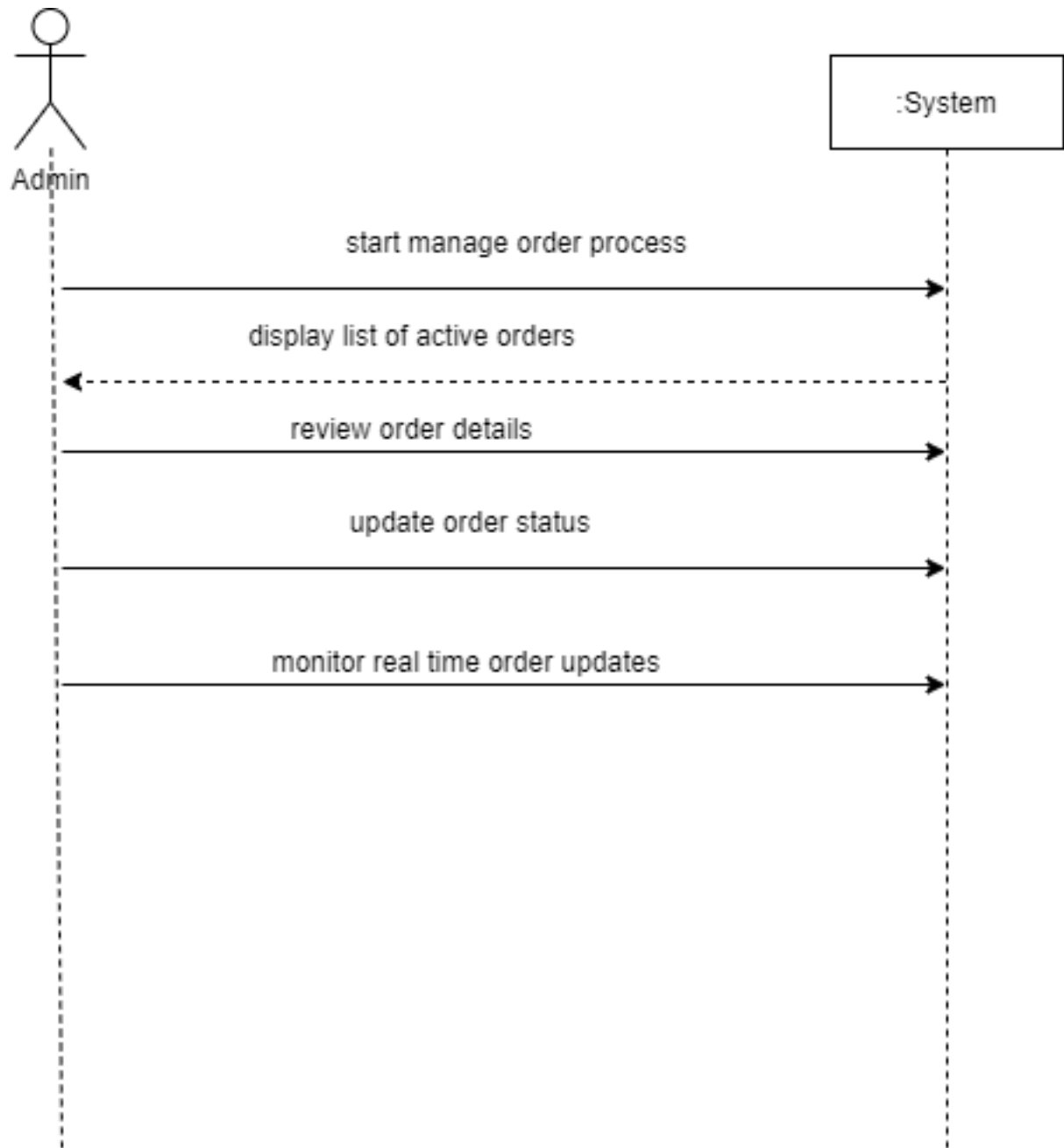


Figure A.8 Manage order

9. Manage menu

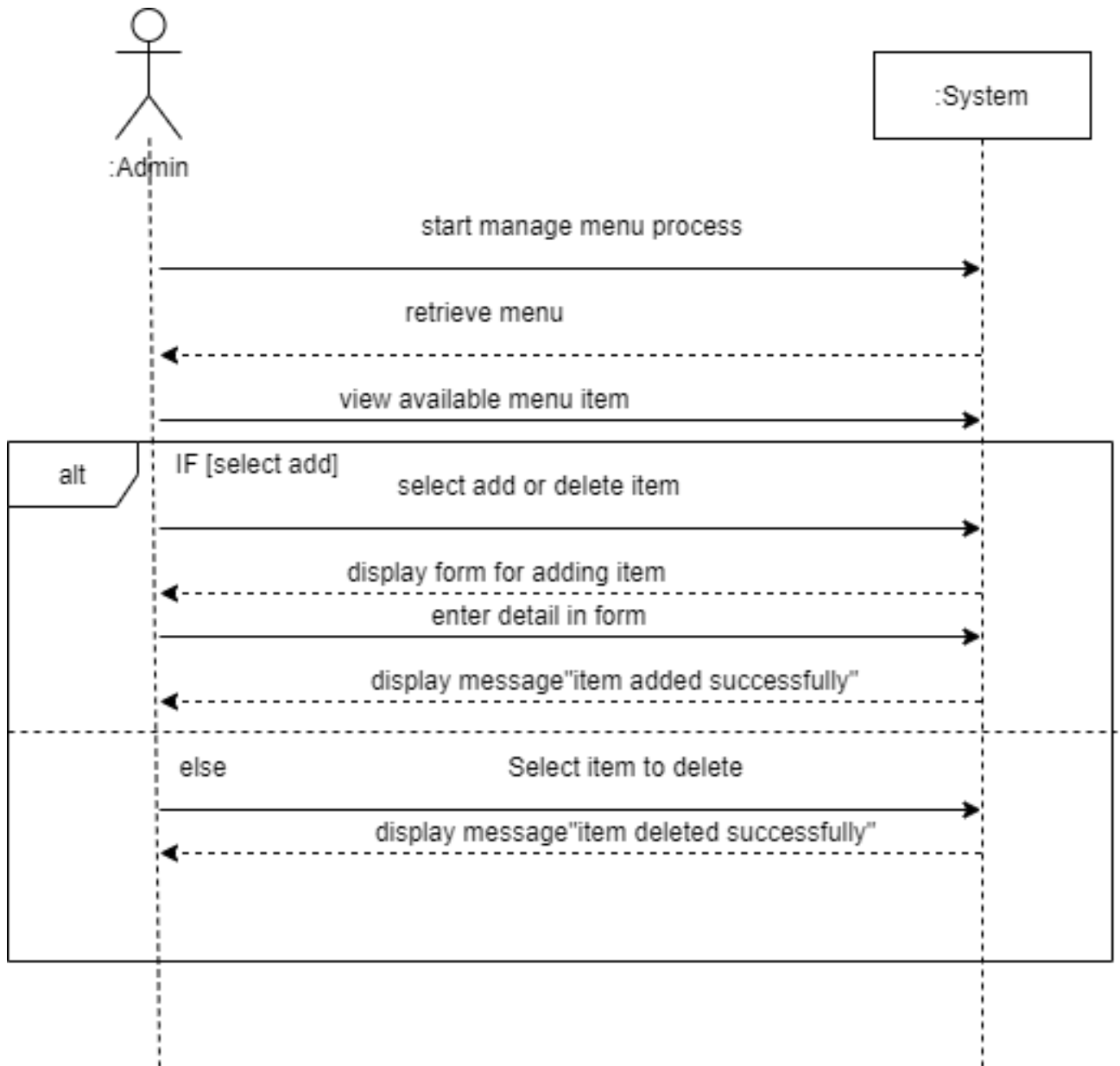


Figure A.9 Manage menu

10.Faculty order

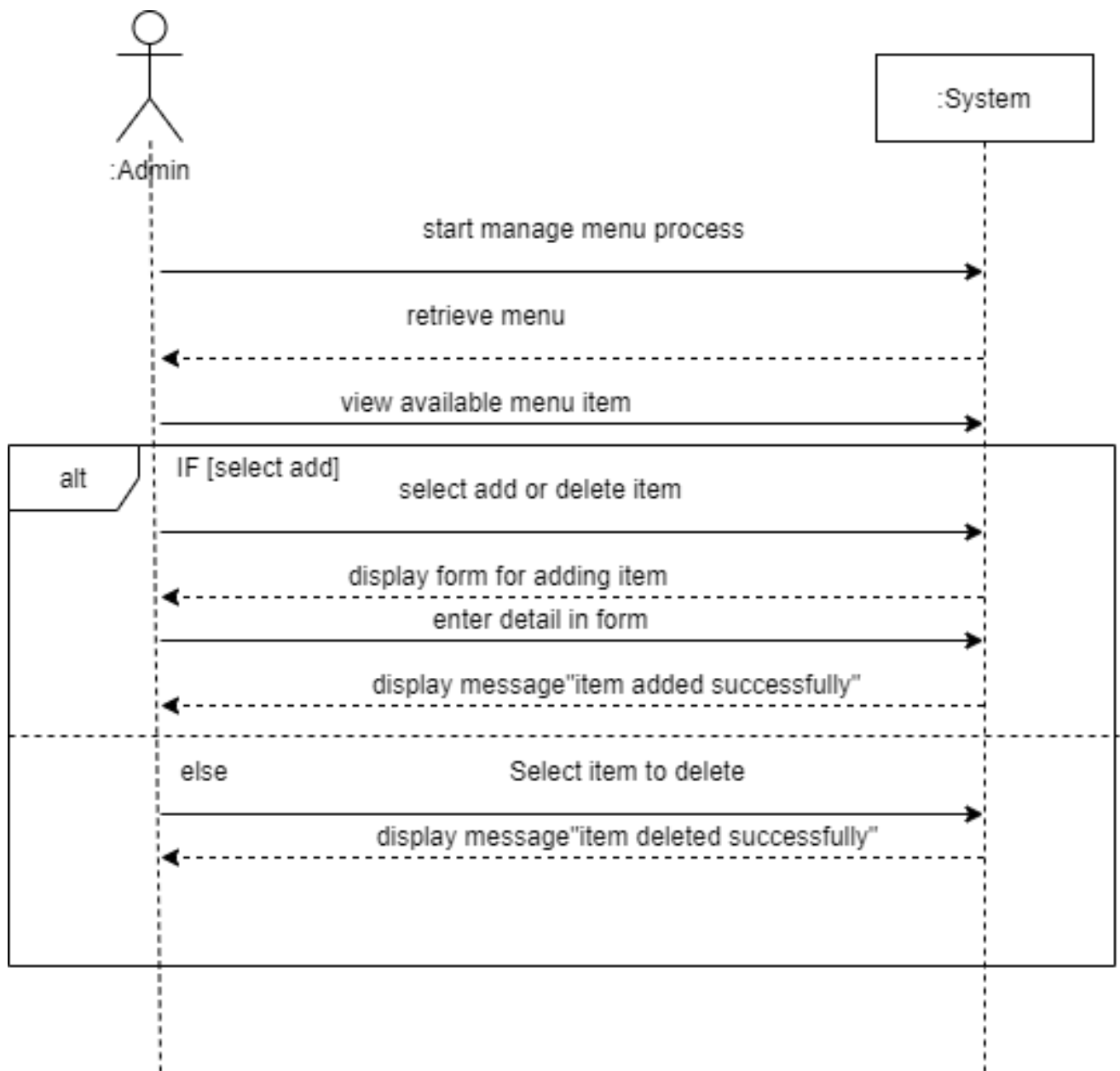


Figure A.10 Faculty order

11.Bulk order

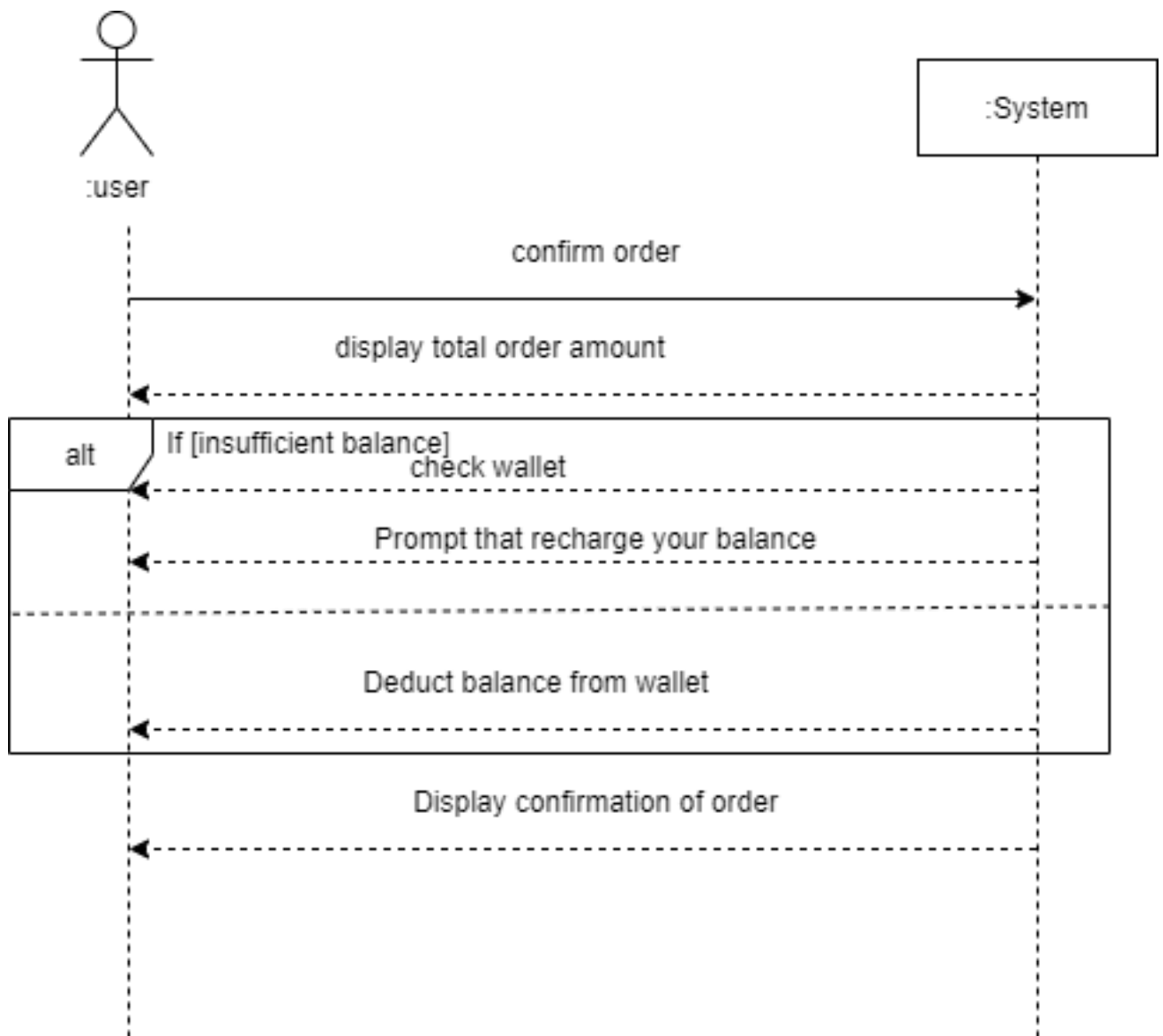


Figure A.11 Place bulk-order

12.Deliver order

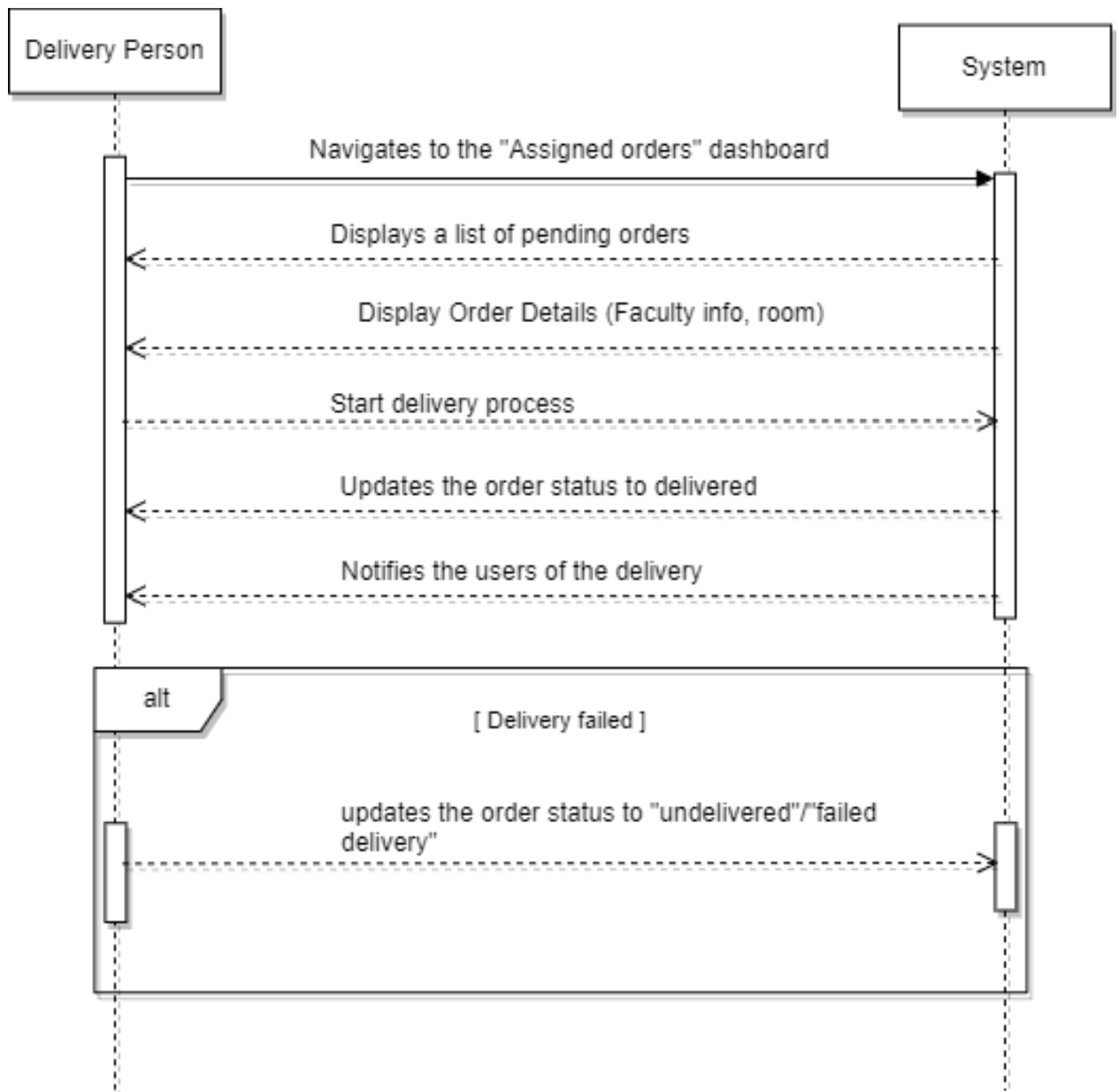


Figure A.12 Deliver order

Appendix B:

Activity Diagram

1. Signup

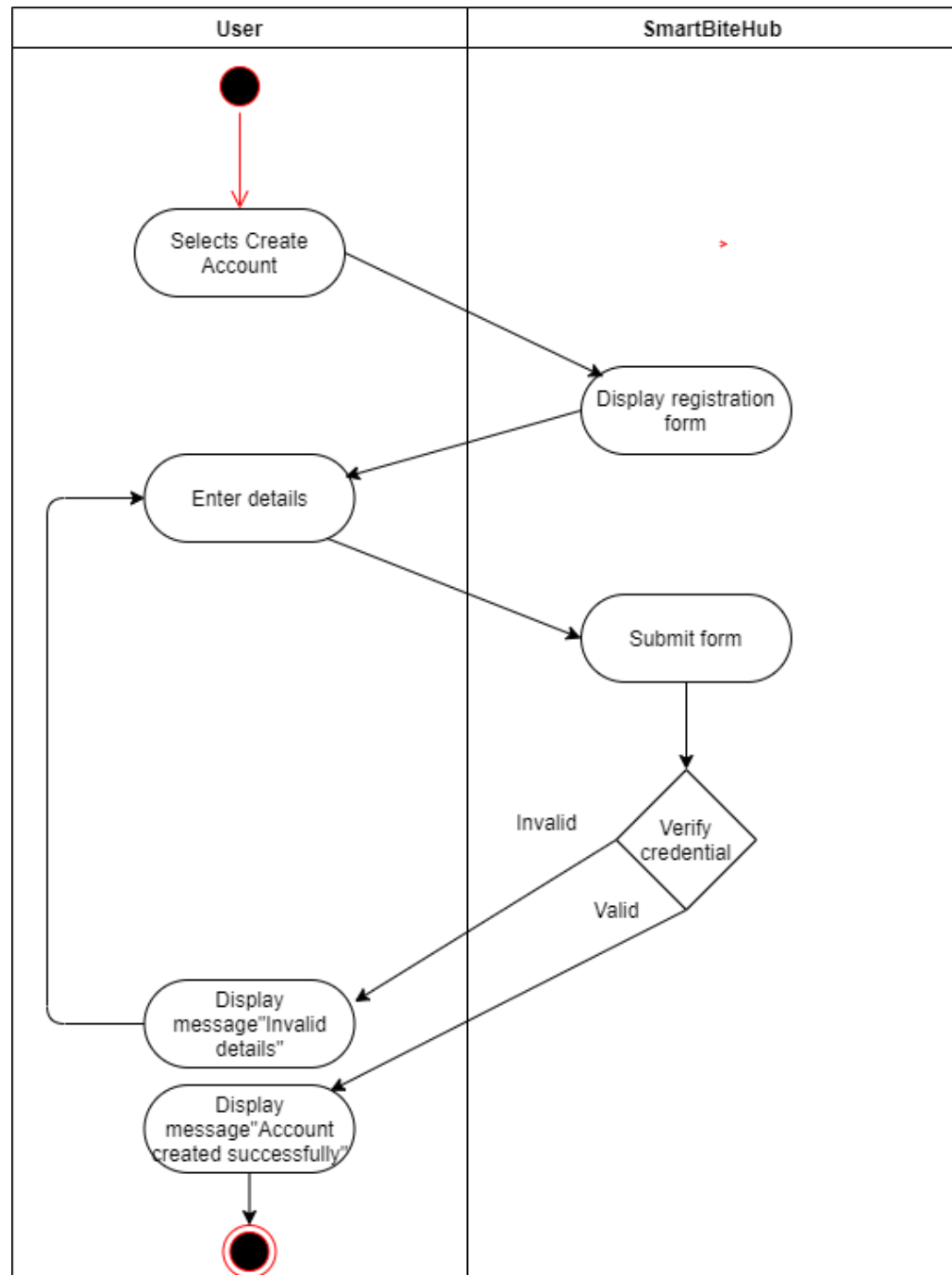


Figure. B.1 Signup

2. Login

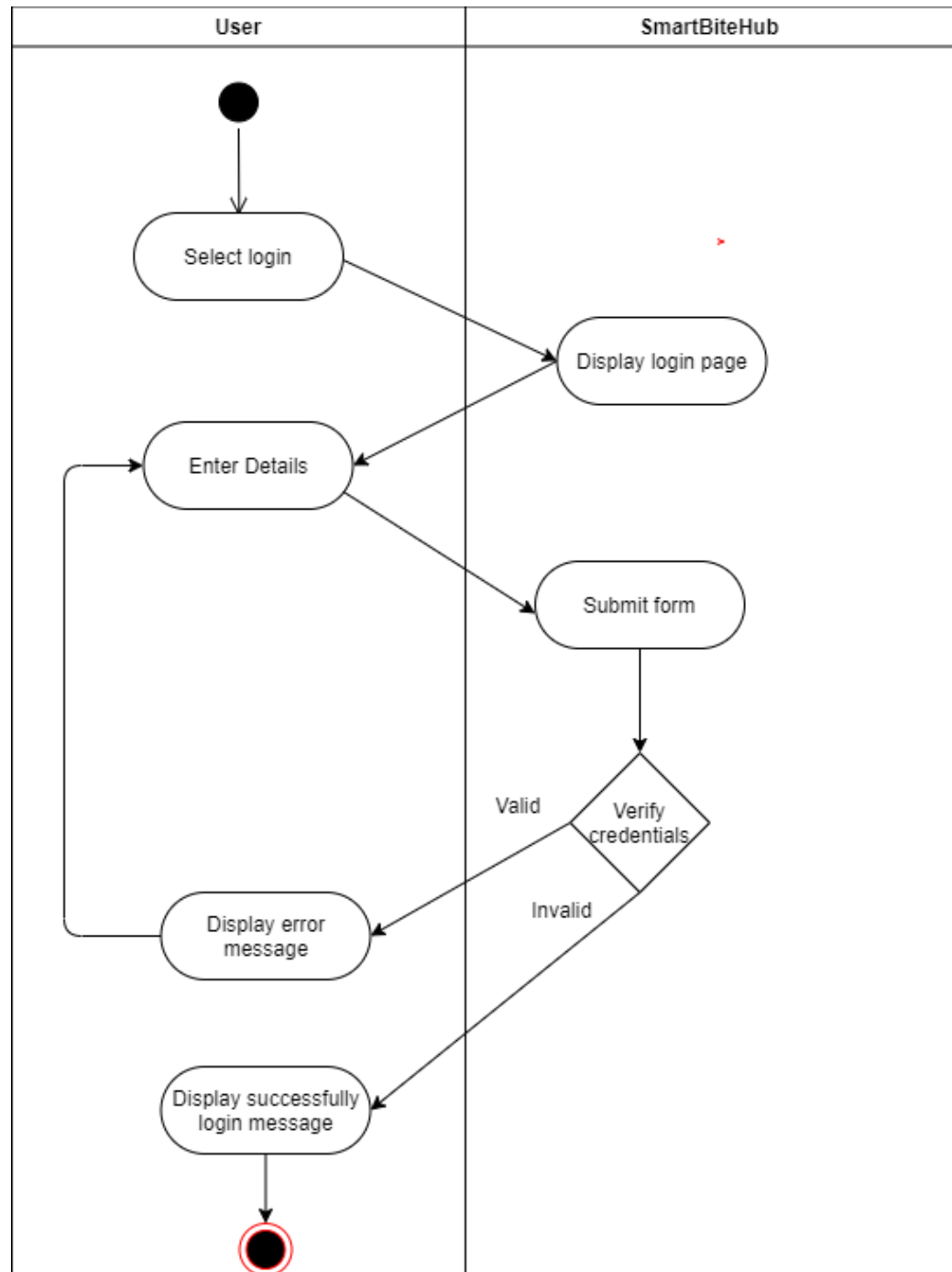


Figure B.2 Login

3. Place order

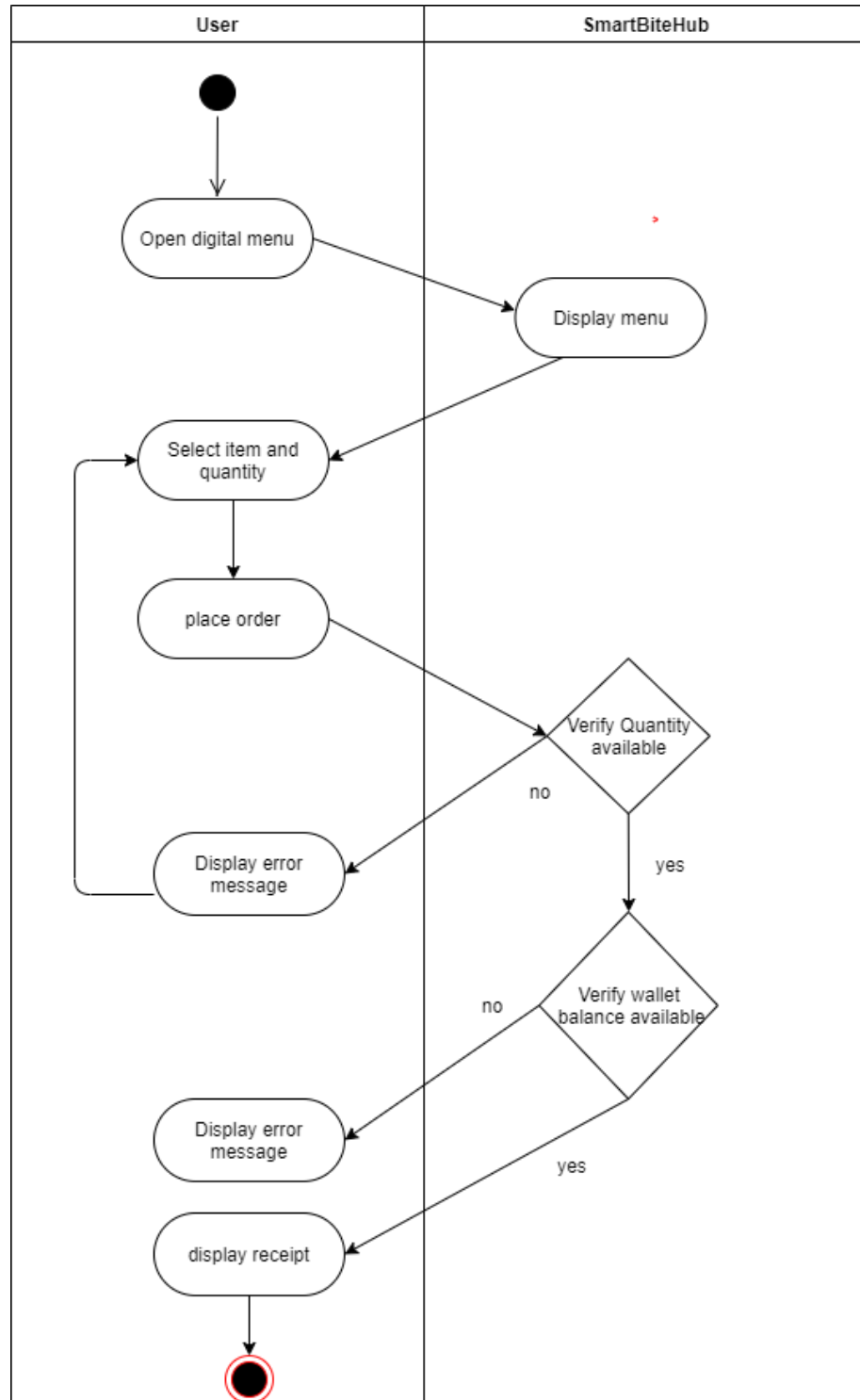


Figure B.3 Place order

4. Provide feedback

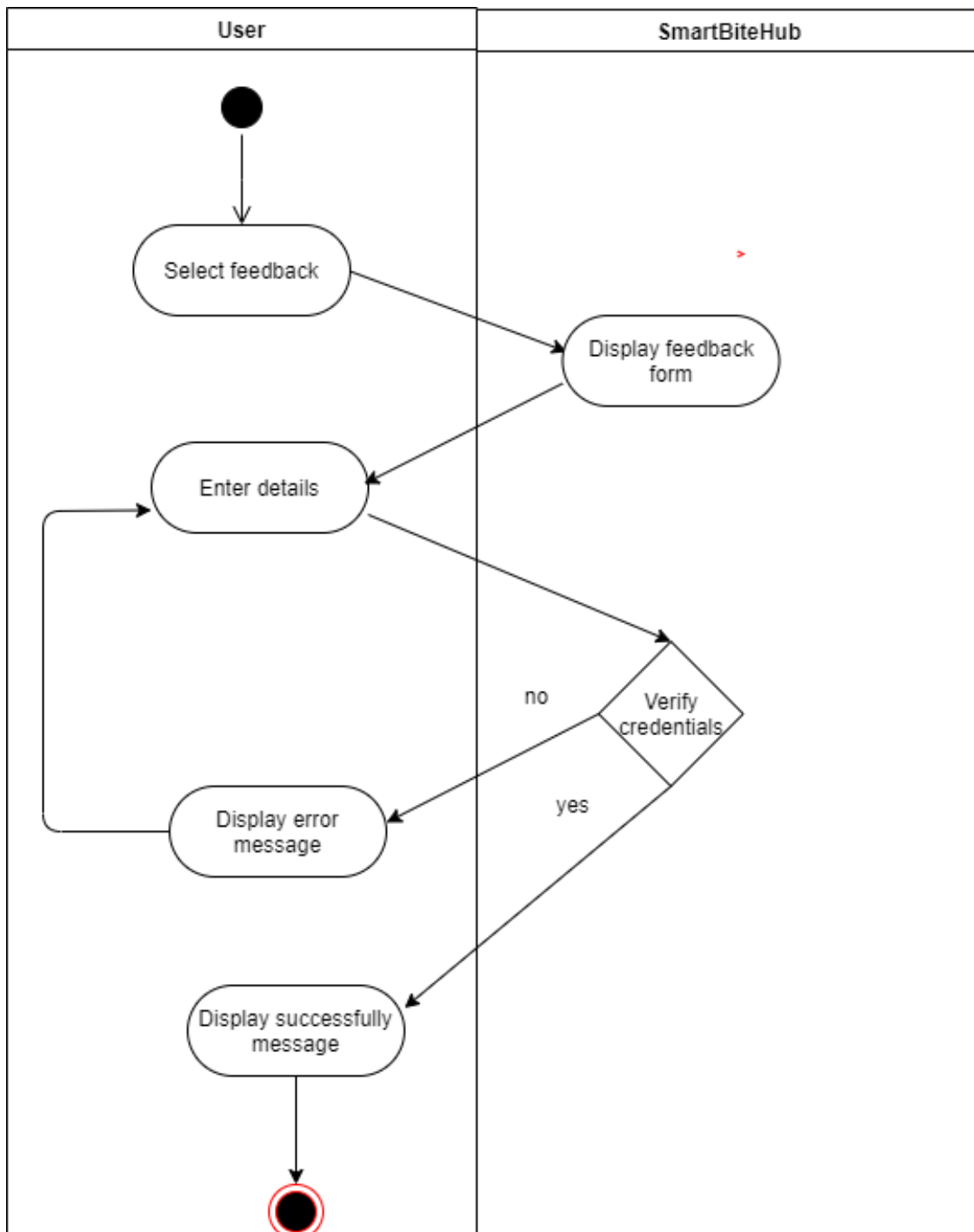


Figure B.4 Provide feedback

5. Manage order

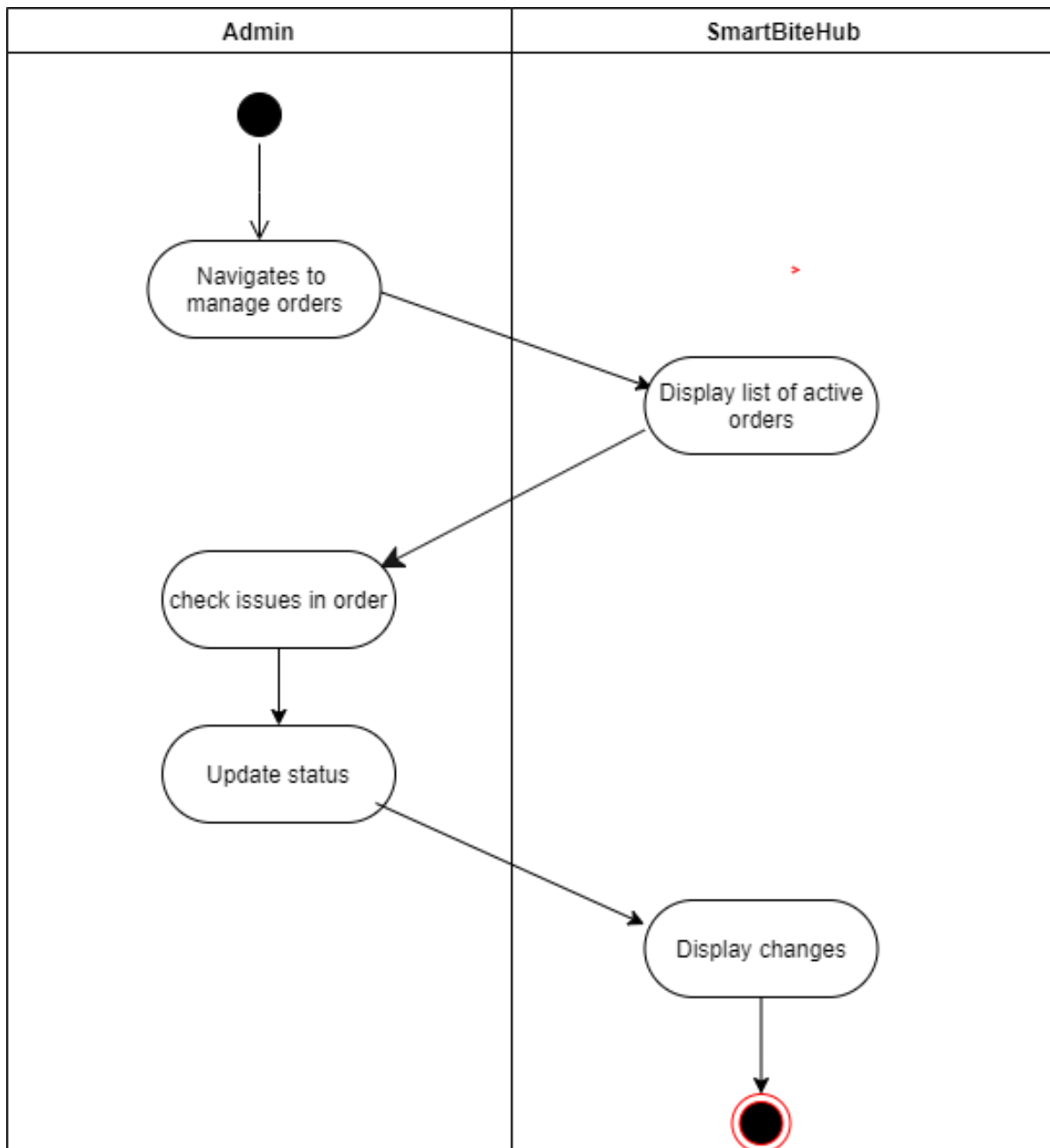


Figure B.5 Manage order

6. Schedule bulk order

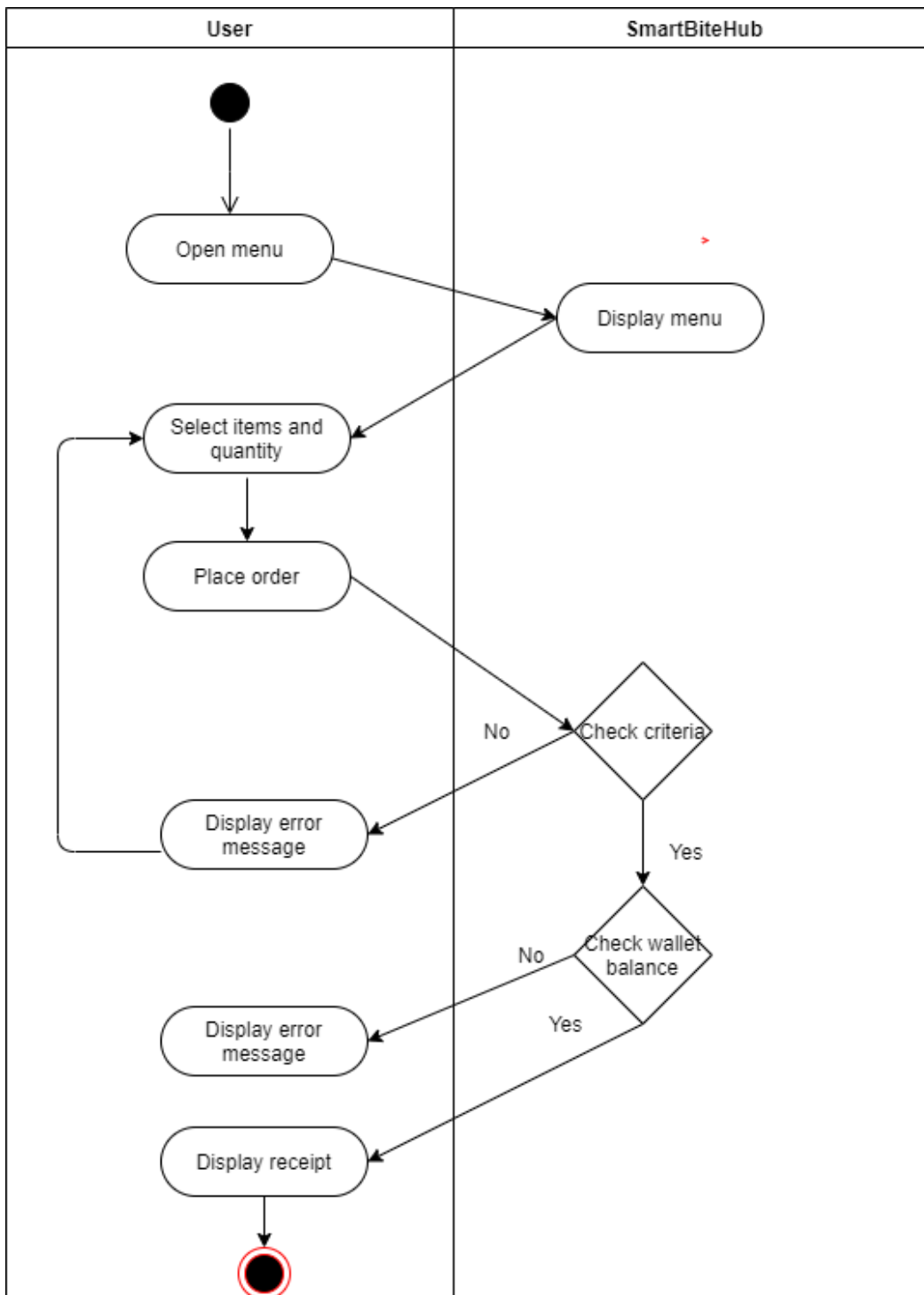


Figure B.6 Schedule bulk order

7. Manage records

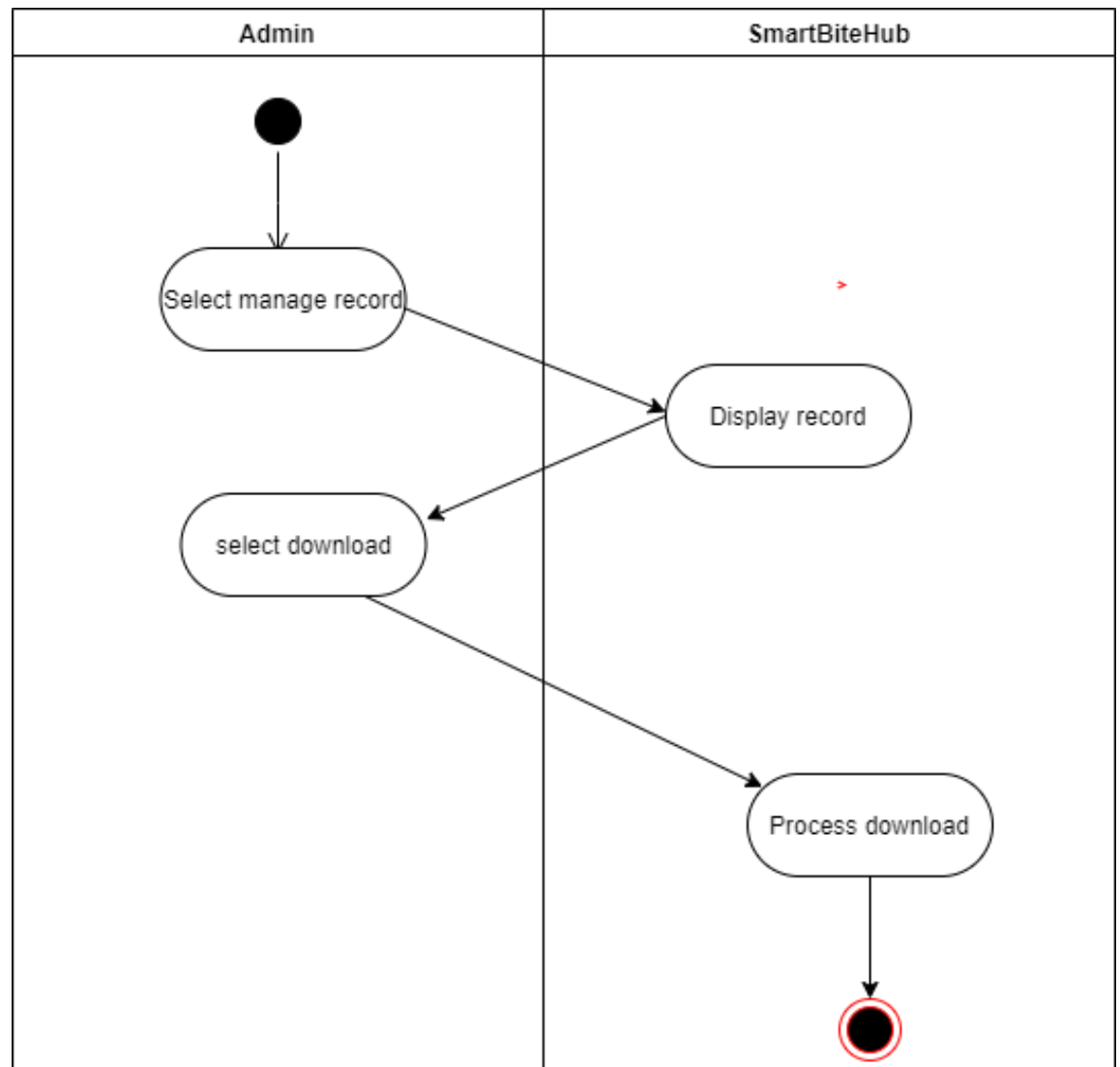


Figure B.7 Manage Receipt

8. Schedule Pre-order

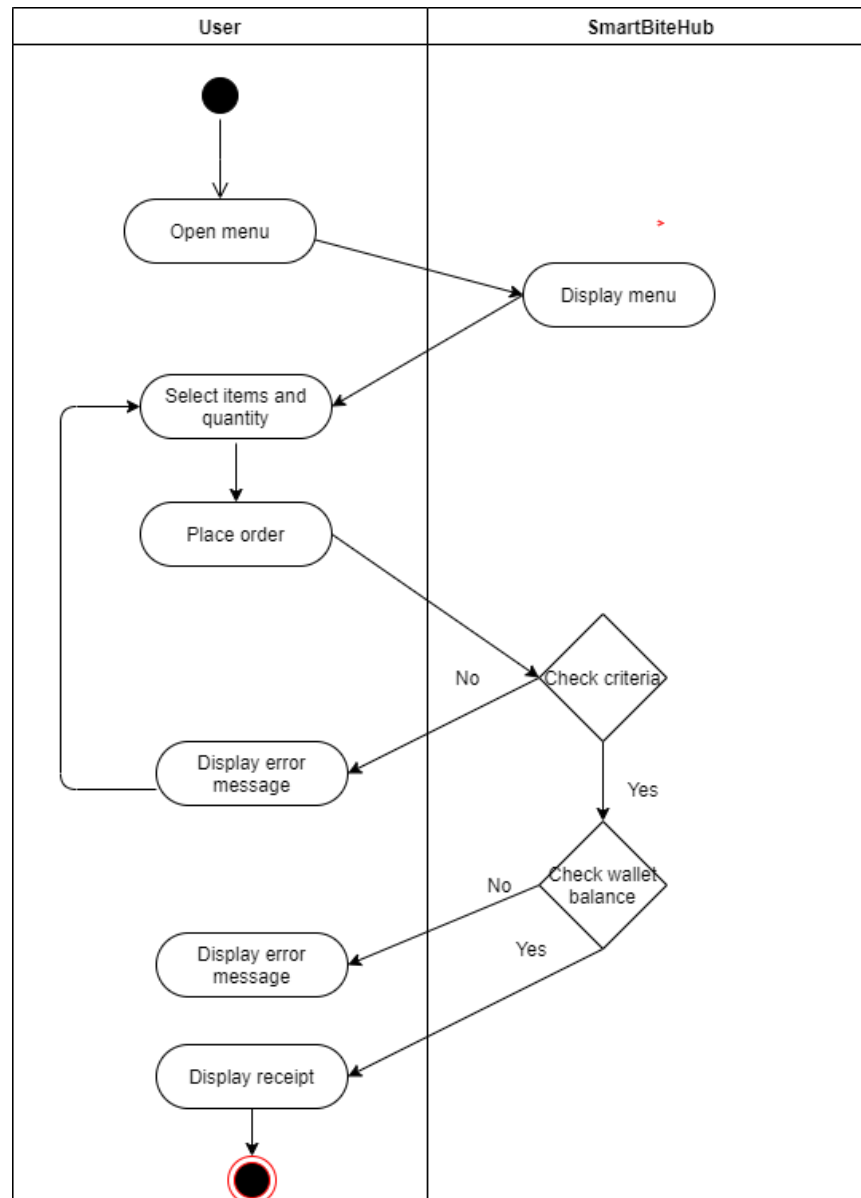


Figure B.8 Schedule Pre-order

9. Special menu

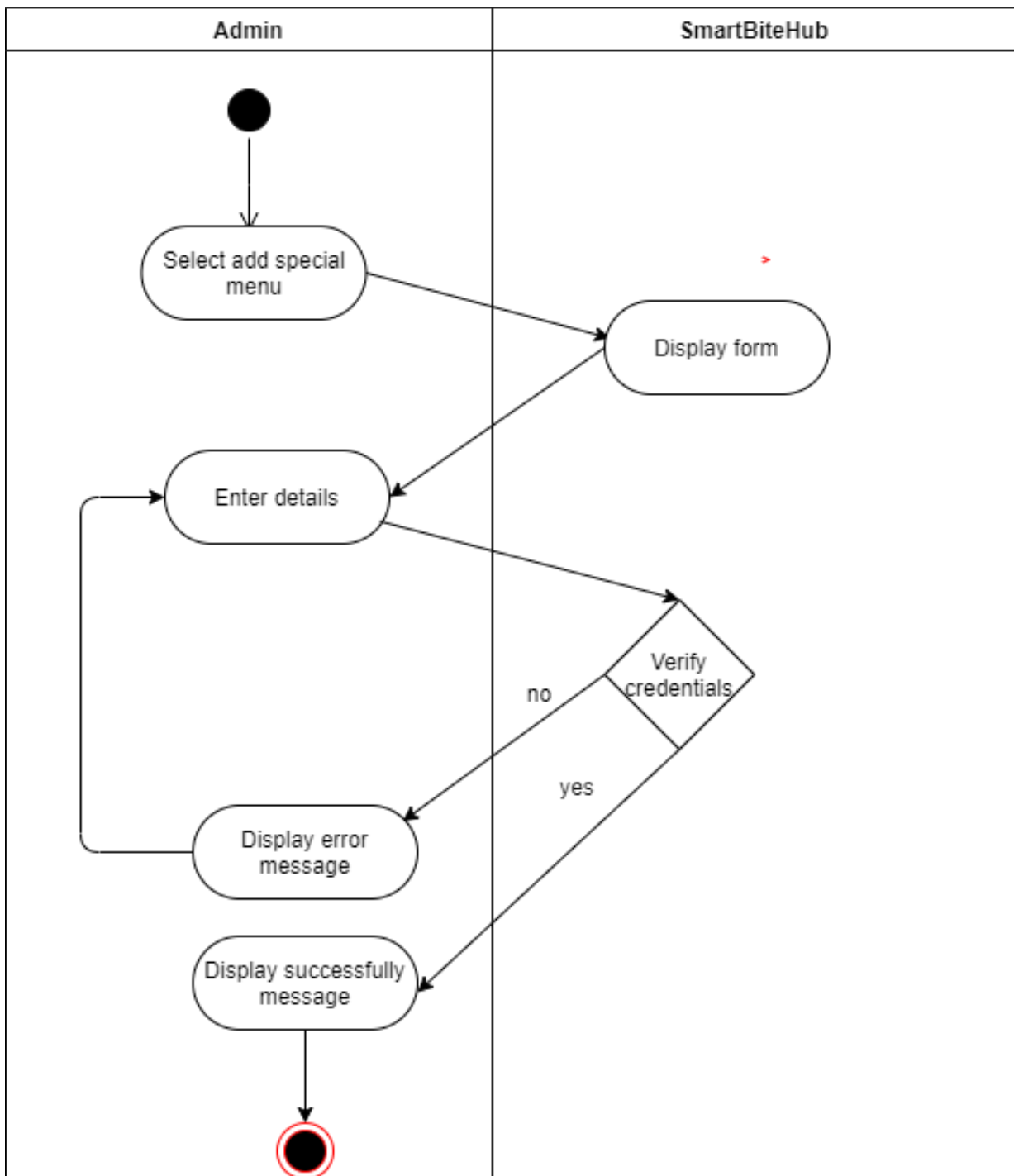


Figure B.9 Special menu

10. Regular menu

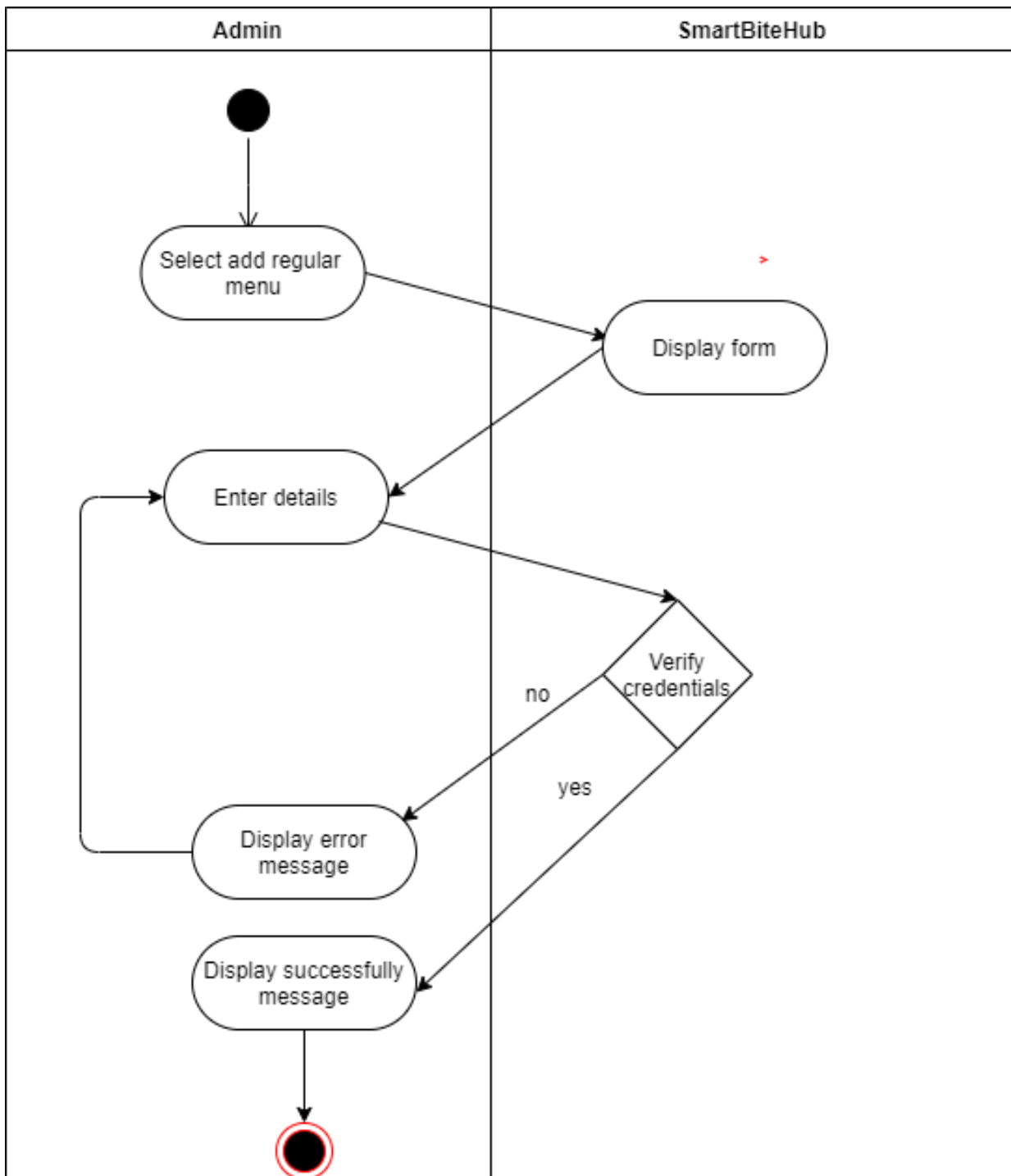


Figure B.10 Regular menu

11. Daily menu

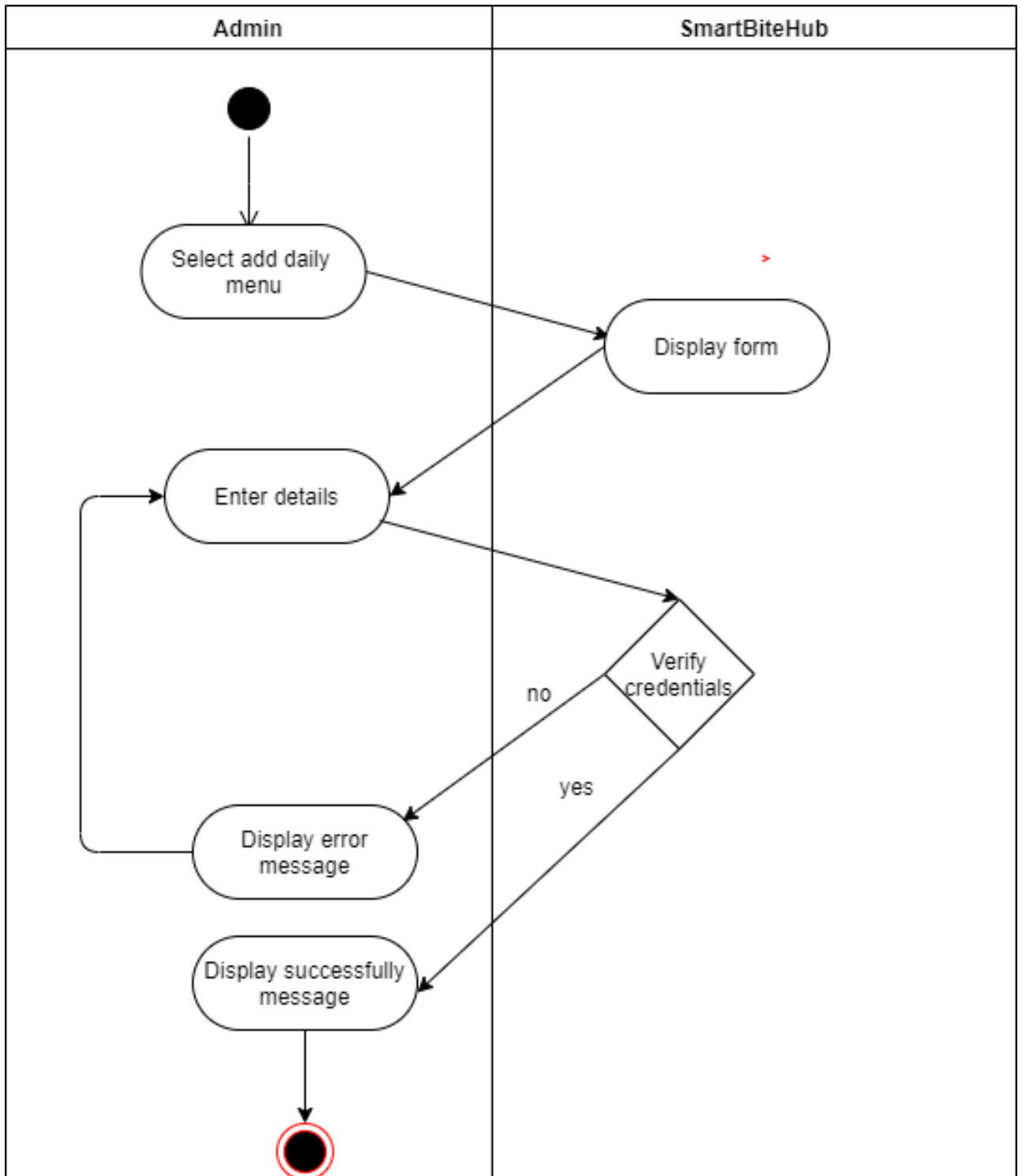


Figure B.11 Daily menu

12. Recharge wallet

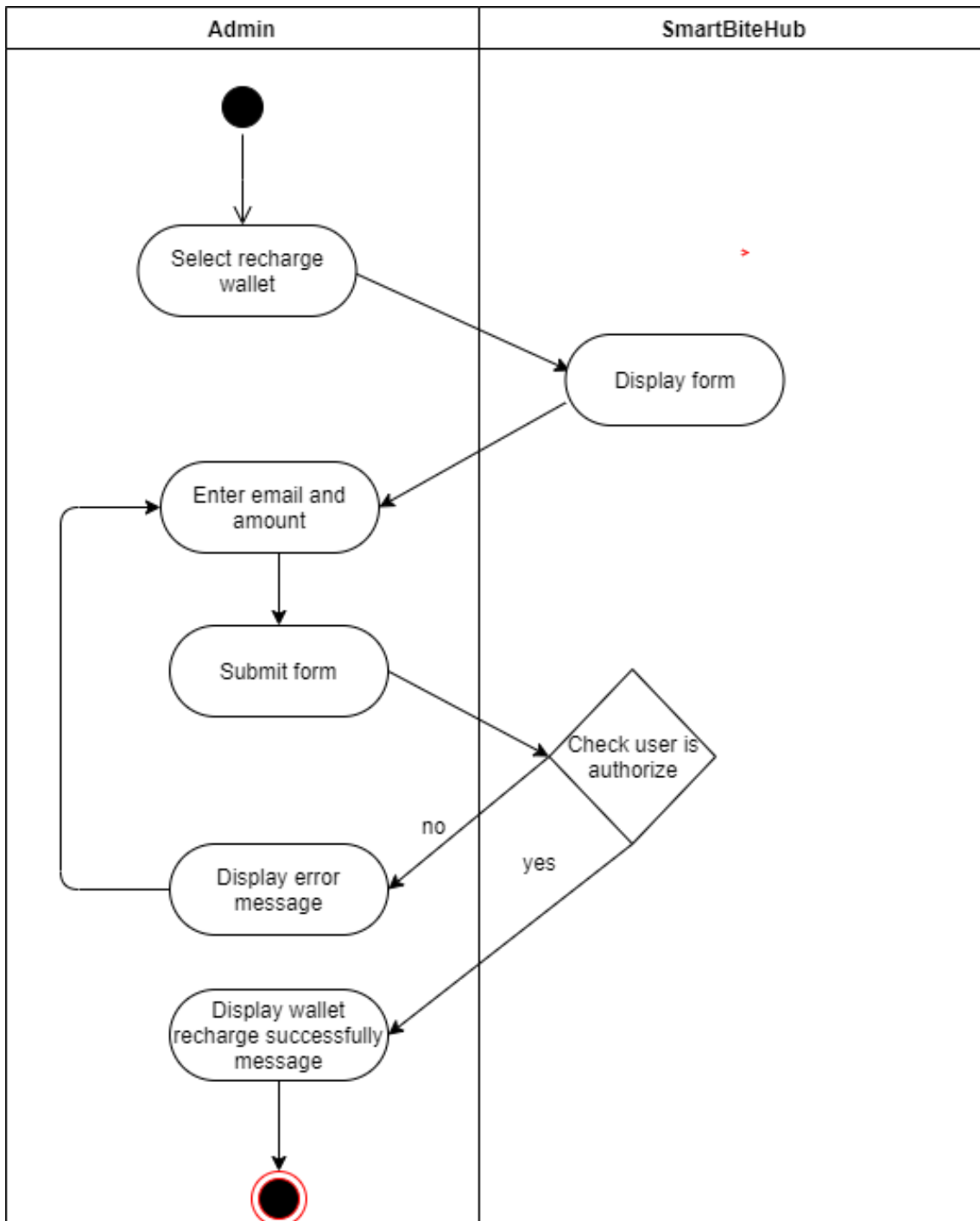


Figure B.12 Recharge wallet

13. Deliver order

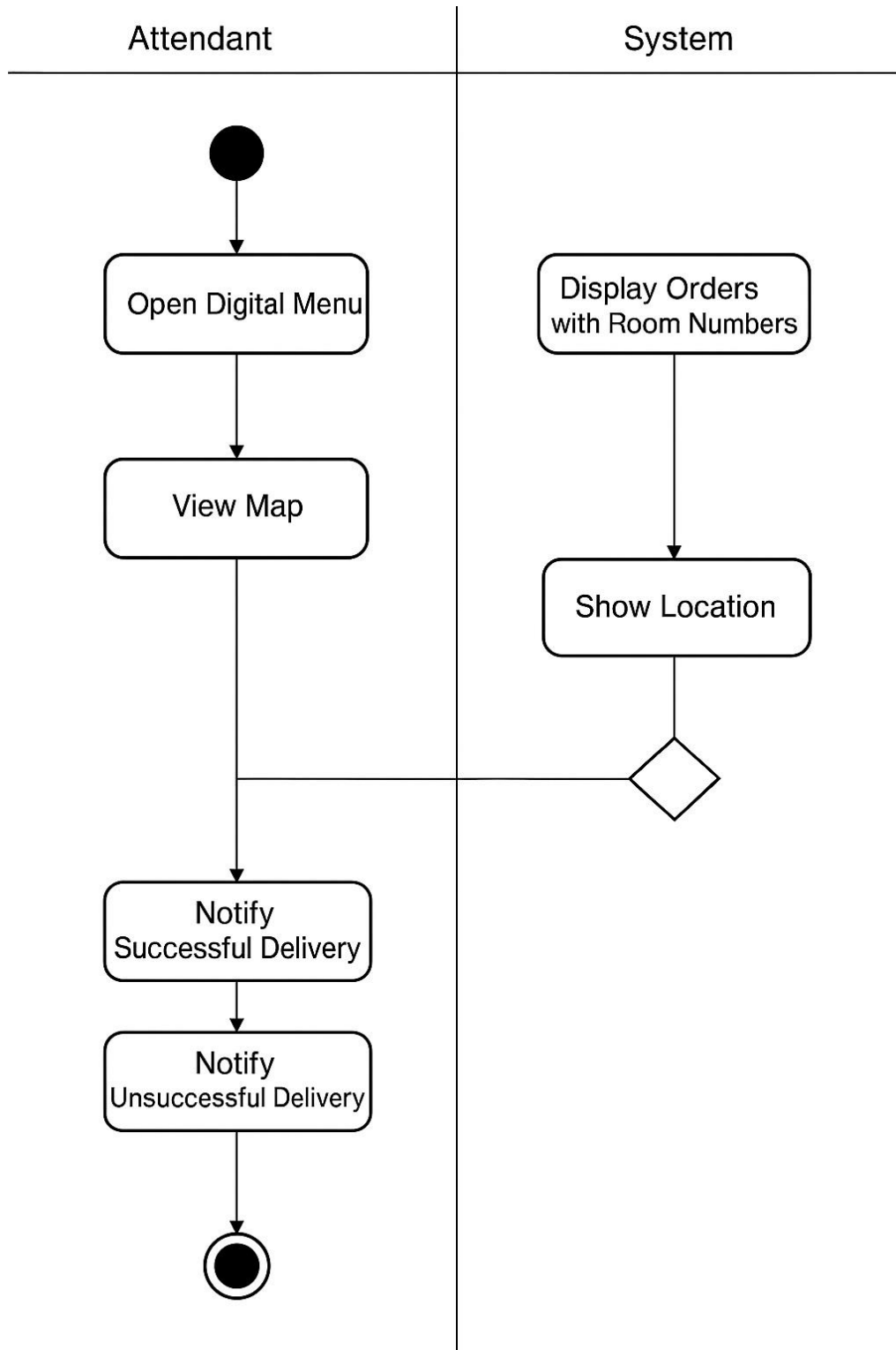


Figure B.13 Deliver order

14. Cancel order

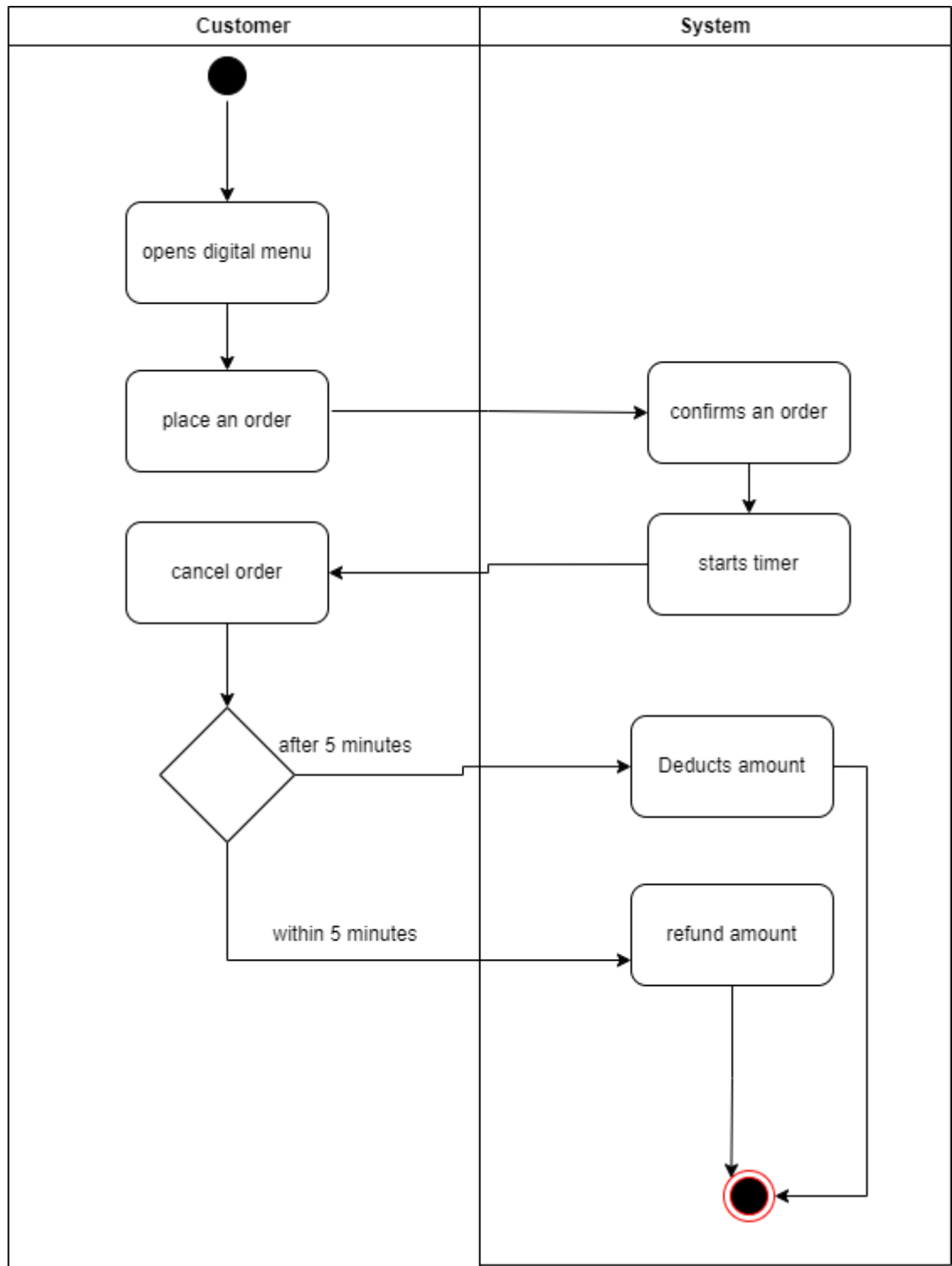


Figure B.14 Cancel order

Appendix C: Test Scenarios

1. ECP and BVA:

Equivalence Classes	Input Value	Valid/Invalid
Username contains only letters (A-Z, a-z)	Alishba Javed	Valid
Username length $3 \leq \text{length} \leq 15$	Alishba Javed	Valid
Username length < 3	Al	Invalid
Username length > 15	khfrybhtfvghjuio	Invalid
Username contains special characters (-, _, \$, %, etc.)	Alishba_javed	Invalid
Username contains numbers (0-9)	Alishba123	Invalid

Table 9.9 ECP

Boundary	Input Value	Valid/Invalid
Username length = 3	Ali	Valid
Username length = 4	Hira	Valid

Username length = 5	Emaan	Valid
Username length = 14	Alishbaah Javeed	Valid
Username length = 15	Alishbaah Javaid	Valid
Username length = 16	Alishba Javed Iqba	Invalid

Table 10.10 BVA

Equivalence Classes	Input Value	Valid/Invalid
Valid email format	alishba@ gmail.com	Valid
Invalid Email Format	hira@.com	Invalid
Invalid Email Format	hira123@ gmail	Invalid
Invalid Email Format	alishbagmailcom	Invalid

Table 11.11 ECP

Equivalence Classes	Input Value	Valid/Invalid
8 >= Password Length <= 12	Alishba123	Valid
Password Length < 8	hira123	Invalid

Password Length >12	hira1234567890123	Invalid
Password = {1,2,3,4, ...}	12345678	Invalid
Password = (Aa-Zz) + {1,2,3,..} +{@#\$%^&....}	Alishba1234@#	Valid

Table 12.12 BVA

Boundary	Input Value	Valid/Invalid
Password Length = 7	Hir%89	Invalid
Password Length = 8	Hira1234	Valid
Password Length = 9	Hira89876	Valid
Password Length = 11	Alishba123#	Invalid
Password Length = 12	Alishba123#%	Valid
Password Length = 13	Alishba127\$0u@	Invalid

Equivalence Classes	Input Value	Valid/Invalid
comment<= 250	250	Valid
comment<250	249	Valid
Comment>250	251	Invalid

Table 13.12 ECP

Boundary	Input Value	Valid/Invalid
Comment Length= 249	Dhsfg249	Invalid
Comment Length= 250	Dhsfg250	Valid
Comment Length= 251	Dhsfg251	inValid

Table 14.13 BVA

Test Case ID	Scenario/Condition	Username	Address	Email	Password	Expected Result
Registration-1	Scenario 1- successful registration	V	V	V	V	User registered successfully
Registration-2	Scenario 2- Invalid username	I	V	V	V	Error message; back to registration screen.
Registration-3	Scenario 3- Invalid email	V	I	V	V	Error message; back to registration screen.
Registration-4	Scenario 4- Invalid password	V	V	V	I	Error message; back to registration screen.
Registration-5	Scenario 4- Invalid address	V	I	V	V	Error message; back to registration screen.
Registration-6	Scenario 5- Invalid address	V	V	V	N/A	Error message; back to registration screen.

Registration -7	Scenario 6- Invalid username	N/A	V	V	V	Error message; back to registration screen.
Registration -8	Scenario 7- Invalid email	V	V	N/A	V	Error message; back to registration screen.
Registration -9	Scenario 8- Invalid password	V	V	V	N/A	Error message; back to registration screen.

Table 15.13 Testcase Criteria

Testcase Scenario:

Test Case-Id	Requirements Description	Test Steps	Test Data	Expected Result
TC-01	The system shall verify the user's information using the following criteria such as the email must be in the format of @gmail.com, the	Enter a valid email. Enter a valid username. Enter a valid password.	Email: alishba@gmail.com Username: Alishba Password: abcr12345	User registered successfully

	username must consist of alphabetic characters and be between 3 to 15 characters in length, and the password must contain numeric and alphabetic characters and special character and be between 8 to 12 characters in length.	Click register button		
TC-02	If user attempts to register with empty fields.	Leave all fields blank. Click register button	Email: [blank] Username: [blank] Password: [blank]	The system should display error messages indicating that “All fields are required.”
TC-03	If the user enters an incorrect password.	Enter a valid email. Enter a valid username. Enter a invalid password. Click register button	Email: alishba@gmail.com Username: Alishba Password: dghytbn	System displays an error message to the user “incorrect password”.

TC-04	If user attempts to register with an invalid email format.	Enter a invalid email. Enter a valid username. Enter a valid password. Click register button	Email: alishba@.com Username: Alishba Password: abcr12345	System displays an error message to the user “incorrect email”.
TC-05	If user attempts to register with an invalid email format, invalid password and invalid name.	Enter a invalid email. Enter a invalid username. Enter a invalid password. Click register button	Email: alishba@.com Username: Alishba Password: abc	System displays an error message to the user “incorrect email ,password and name”.
TC-05	If user attempts to register with an valid email format, valid password and invalid name.	Enter a valid email. Enter a invalid username. Enter a valid password. Click register button	Email: alishba@gmail.com Username: Alishba123 Password: abcr12345	System displays an error message to the user “incorrect name”.

TC-06	User enter email and password to login into the system	Enter a valid email. Enter a valid password. Click Login button	Email: alishba@gmail.com Password: abcr12345	User login successfully and dashboard appear.
TC-07	User enter invalid email and password to login into the system	Enter a invalid email. Enter a valid password. Click Login button	Email: alishbagmail.com Password: abcr12345	System displays an error message to the user “incorrect email”.
TC-08	User enter email and invalid password to login into the system	Enter a valid email. Enter a invalid password. Click Login button	Email: alishba@gmail.com Password: abc	System displays an error message to the user “incorrect password”.
TC-09	User enter blank email and password to login into the system	Leave all fields blank. Click Login button	Email: [blank] Username: [blank] Password: [blank]	The system should display error messages indicating that “All fields are required.”

TC-10	User enter invalid email and invalid password to login into the system	Enter a invalid email. Enter a invalid password. Click Login button	Email: alishbagmail.com Password: abc	System displays an error message to the user “incorrect email and password”.
TC - 11	User enter unregistered email and password to login into the system	Enter a unregistered email. Enter a invalid password. Click Login button	Email: alishba@gmail.com Password: abcr12345	System displays an error message to the user “User doesn’t exist”.
TC-12	User give feedback by providing accurate details such as email, name and comment must be maximum of 250 words.	Enter valid email Enter valid name Enter valid comment Click Submit button	Email: alishba@gmail.com Name: Hira Comment: Great!	System displays an error message to the user “Feedback submitted successfully”.
TC-13	User give feedback by providing details such as invalid email, name and comment must be maximum of 250 words.	Enter invalid email Enter valid name Enter valid comment	Email: alishbagmail.com Name: Hira Comment: Great!	System displays an error message to the user

		Click Submit button		“incorrect email”.
TC-14	User give feedback by providing details such as email, invalid name and comment must be maximum of 250 words.	Enter valid email Enter invalid name Enter valid comment Click Submit button	Email: alishba@gmail.com Name: Hira123 Comment: Great!	System displays an error message to the user “incorrect name”.
TC-15	User give feedback by leaving all fields empty such as email, name and comment must be maximum of 250 words.	Leave all fields blank. Click Submit button	Email: [blank] Name: [blank] Comment: [blank]	The system should display error messages indicating that “All fields are required.”
TC-16	User give feedback by providing details such as email, name and invalid comment must be maximum of 250 words.	Enter valid email Enter valid name Enter invalid comment Click Submit button	Email: alishba@gmail.com Name: Hira123 Comment	The system should display error messages indicating that “All fields are required.”
TC-17	User give feedback by providing details such as email, name	Enter valid email	Email: alishba@gmail.com Name: Hira123	The system should display error

	and comment increases maximum limit of 250 words	Enter valid name Enter invalid comment limit Click Submit button	Comment:sjdghafd300 words	messages indicating that “Comment exceeds maximum character limit.”
TC-18	Admin shall add items in menu by providing name, description, price , quantity and upload image of the food.	Enter valid Name Enter valid Price Enter valid Description Enter valid quantity Enter image Click Add button	Name: "Pasta" Price: 10 Description: "Delicious pasta" Quantity:10 Image:food.png	Menu added successfully
TC-19	Admin shall add items in menu by providing all the fields blank including name, description, price , quantity and upload image of the food.	Leave all fields blank. Click Add button	Name: [blank] Price: [blank] Description: [blank] Quantity:10 Image: [blank]	The system should display error messages indicating that “All fields are required.”
TC-20	Admin shall add items in menu by providing empty name, description,	Enter empty Name Enter valid Price	Name: [blank] Price: 10 Description: "Delicious pasta"	The system should display error messages

	price , quantity and upload image of the food.	Enter valid Description Enter valid quantity Enter image Click Add button	Quantity:10 Image:food.png	indicating that “All fields are required.”
TC-21	Admin shall add items in menu by providing name, description, empty price , quantity and upload image of the food.	Enter valid Name Enter empty Price Enter valid Description Enter valid quantity Enter image Click Add button	Name: “Pasta” Price: [blank] Description: "Delicious pasta" Quantity:10 Image:food.png	The system should display error messages indicating that “All fields are required.”
TC-22	Admin shall add items in menu by providing name,empty description, price , quantity and upload image of the food.	Enter valid Name Enter valid Price Enter empty Description Enter valid quantity Enter image Click Add button	Name: “Pasta” Price: 10 Description: [blank] Quantity:10 Image:food.png	The system should display error messages indicating that “All fields are required.”

TC-23	Admin shall add items in menu by providing name, description, price , empty quantity and upload image of the food.	Enter valid Name Enter valid Price Enter valid Description Enter empty quantity Enter image Click Add button	Name: "Pasta" Price: 10 Description: "Delicious pasta" Quantity: [blank] Image:food.png	The system should display error messages indicating that "All fields are required."
TC-24	Admin shall add items in menu by providing name, description, price , quantity and upload empty image of the food.	Enter valid Name Enter valid Price Enter valid Description Enter valid quantity Enter empty image Click Add button	Name: "Pasta" Price: 10 Description: "Delicious pasta" Quantity: 10 Image: : [blank]	The system should display error messages indicating that "All fields are required."
TC-25	Admin shall recharge wallet of the user by entering their registered email and amount in their wallets.	Enter valid email Enter valid amount Click recharge button	Email: alishba@gmail.com Amount:10	Recharge successfully

TC-26	Admin shall recharge wallet of the user by entering their unregistered email and amount in their wallets.	Enter invalid email Enter valid amount Click recharge button	Email: alishbagmail.com Amount:10	System displays an error message to the user "incorrect email".
TC-27	Admin shall recharge wallet of the user by entering all the fields blank including their registered email and amount in their wallets.	Leave all fields blank. Click recharge button	Email: [blank] Amount:[blank]	The system should display error messages indicating that "All fields are required."
TC-28	Admin shall recharge wallet of the user by entering their registered email and zero amount in their wallets.	Enter valid email Enter zero amount Click recharge button	Email: alishba@gmail.com Amount:0	The system should display error messages indicating that "Invalid recharge amount."
TC-29	Admin shall recharge wallet of the user by entering their registered email and negative amount in their wallets.	Enter valid email Enter negative amount Click recharge button	Email: alishba@gmail.com Amount:-1	The system should display error messages indicating that "Invalid

				recharge amount."
TC-30	Admin shall recharge wallet of the user by entering their unregistered email and amount in their wallets.	Enter unregistered email Enter valid amount Click recharge button	Email: <u>alishb@gmail.com</u> Amount:10	The system should display error messages indicating that "wallet doesn't exist".
TC-31	User shall enter correct otp to reset their password.	Enter valid OTP within 15minute timer navigate to new password page	OTP: 123456	OTP matches navigate to the new password page
TC-32	User shall enter incorrect otp to reset their password.	Enter invalid OTP	OTP: 654321	The system should display an error message: "Invalid OTP. Please try again."
TC-33	User shall not enter otp within 15 minutes timer to reset their password.	Enter valid OTP after 15-minute timer expires	OTP: 123456	The system should display an error

				message: "OTP has expired. Please request a new one."
TC-34	User shall enter correct otp to reset their password.	Enter valid email for OTP Click submit button	Email: alishba@gmail.com	Navigates to otp timer screen
TC-35	User shall enter incorrect email to receive otp to reset their password.	Enter invalid email for OTP Click submit button	Email: alishbagmail.com	System display message "Invalid email".
TC-36	User shall enter empty otp to reset their password.	Enter empty email for OTP Click submit button	Email: [blank]	System display message "Invalid email".
TC-37	User shall enter correct new password and confirm password must be matched to reset their password successfully.	Enter valid new password Enter valid confirm password Click submit button	New Password: abcr12345 Confirm Password: abcr12345	System display message "Password changed successfully" and navigates to login page.

TC-38	User shall enter correct new password and invalid confirm password must be matched to rest their password successfully.	Enter valid new password Enter invalid confirm password Click submit button	New Password: abcr12345 Confirm Password: abcr123	System display error message "Passwords do not match."
TC-39	User shall enter correct new password and empty confirm password must be matched to rest their password successfully.	Enter valid new password Enter empty confirm password Click submit button	New Password: abcr12345 Confirm Password:	The system should display error messages indicating that "All fields are required."
TC-40	User shall enter empty new password and empty confirm password must be matched to rest their password successfully.	Enter empty new password Enter empty confirm password Click submit button	New Password: Confirm Password:	The system should display error messages indicating that "All fields are required."
TC-41	Admin shall recharge wallet with non-numeric amount.	Enter valid email, enter non-numeric amount, click recharge.	Email: alishba@gmail.com Amount: ten	System displays "Amount must be a numeric value."

TC-42	User places order with sufficient balance.	Login, add item to cart, confirm order.	Balance: 50 Item Price: 10	Order placed successfully, balance updated.
TC-43	User places order with insufficient balance.	Login, add item to cart, confirm order.	Balance: 5 Item Price: 10	Error “Insufficient balance.”
TC-44	User cancels order within 5-minute limit.	Place order, cancel within 5 minutes.	Time: 3 minutes	Order cancelled successfully, balance refunded.
TC-45	User tries to cancel order after 5-minute limit.	Place order, cancel after 5 minutes.	Time: 6 minutes	Error “Cancellation time limit exceeded

Table 16.14 Testcases

Appendix D: Work Breakdown Structure

WBS #	WBS Deliverable	Activity #	Activity to Complete the Deliverable	Duration (# of Days)	Responsible Team Member(s) & Role(s)
1.1	Work Breakdown Structure (WBS)	1.1.1	Define the scope and structure	2 days	Alishba, Hira, Vareesha
1.2	Roles & Responsibility Matrix	1.2.1	Assign roles and document responsibilities	1 day	Alishba, Hira, Vareesha
1.3	Change Control System	1.3.1	Develop process for handling changes	3 days	Alishba, Hira, Vareesha
2.1	Final Documentation Introduction	2.1.1	Write introduction and goals	1 day	Alishba, Hira, Vareesha
2.2	Literature / Market Survey	2.2.1	Conduct research and gather data	10 days	Alishba, Hira, Vareesha
2.3	Requirements Analysis	2.3.1	Gather and analyze requirements	10 days	Alishba, Hira, Vareesha
2.4	System Design	2.4.1	Design system architecture	15 days	Alishba, Hira, Vareesha
2.5	Implementation	2.5.1	Code and implement system	60 days	Alishba, Hira, Vareesha
2.6	Testing & Performance Evaluation	2.6.1	Perform system tests	30 days	Alishba, Hira, Vareesha

2.7	Conclusion & Outlook	2.7.1	Summarize results and future recommendations	5 days	Alishba, Hira, Vareesha
2.8	End User Documentation	2.8.1	Write user-friendly documentation	10 days	Alishba, Hira, Vareesha
2.9	Application Administration Documentation	2.9.1	Write admin documentation	10 days	Alishba, Hira, Vareesha
2.10	System Administrator Documentation	2.10.1	Write system admin documentation	10 days	Alishba, Hira, Vareesha
3.1	Development Environment	3.1.1	Set up IDE, version control, server, database	20 days	Alishba, Hira, Vareesha
3.2	Presentation Layer	3.2.1	Develop user interface	20 days	Alishba, Hira, Vareesha
3.3	Business Logic Layer	3.3.1	Implement business logic	20 days	Alishba, Hira, Vareesha
3.4	Data Management Layer	3.4.1	Implement database and data handling	10 days	Alishba, Hira, Vareesha
3.5	Physical Layer	3.5.1	Set up physical servers and network	20 days	Alishba, Hira, Vareesha

Table 17.15 WBS

Appendix E: Roles & Responsibility Matrix

Roles & Responsibility Matrix:

The purpose of roles & responsibility matrix is to identify who will do what

Sr. No	WBS Deliverable	Responsible Team Member(s) & Role(s)
1.1	Work Breakdown Structure (WBS)	Alishba, Hira, Vareesha
1.2	Roles & Responsibility Matrix	Alishba, Hira, Vareesha
1.3	Change Control System	Alishba, Hira, Vareesha
2.1	Final Documentation Introduction	Alishba, Hira, Vareesha
2.2	Literature / Market Survey	Alishba, Hira, Vareesha
2.3	Requirements Analysis	Alishba, Hira, Vareesha
2.4	System Design	Alishba, Hira, Vareesha
2.5	Implementation	Alishba, Hira, Vareesha
2.6	Testing & Performance Evaluation	Alishba, Hira, Vareesha
2.7	Conclusion & Outlook	Alishba, Hira, Vareesha
2.8	End User Documentation	Alishba, Hira, Vareesha
2.9	Application Administration Documentation	Alishba, Hira, Vareesha
2.10	System Administrator Documentation	Alishba, Hira, Vareesha
3.1	Development Environment	Alishba, Hira, Vareesha
3.2	Presentation Layer	Alishba, Hira, Vareesha
3.3	Business Logic Layer	Alishba, Hira, Vareesha
3.4	Data Management Layer	Alishba, Hira, Vareesha
3.5	Physical Layer	Alishba, Hira, Vareesha

Table 18.16 Roles & Responsibility Matrix

